

操作系统

Principles of Operating System

北京大学计算机科学技术系 陈向群

Department of computer science and
Technology, Peking University

2026 Spring

问题1

- ▶ 基于X86-64的Linux中设计了多大的虚拟地址空间？
- ▶ 虚拟内存有哪些特征？
- ▶ 虚拟内存管理的目标？
- ▶ Linux中如何描述虚拟内存空间（数据结构）？
- ▶ 实现虚拟内存涉及到的数据结构有哪些？
- ▶ 虚拟内存管理中地址转换（地址映射）的作用是什么？
- ▶ 如何加快地址转换过程？
- ▶ 存储体系与虚拟内存空间的关系
- ▶ 页表项（PTE）包括哪些信息？

问题2

- ▶ 页表项（PTE）的哪些控制位会引发Page Fault异常？是硬件发出的还是操作系统发出的？
- ▶ 写时复制（COW）让fork()快速完成，但为什么必须配合"缺页异常"才能实现？如果父子进程都只读，物理页真的被复制了吗？
- ▶ 通过mmap()映射文件时，为什么第一次访问会慢，之后却可能跟访问内存一样快？如果两个进程mmap同一个文件，它们看到的是同一份物理页吗？
- ▶ mmap()与写时复制是什么关系？

虚拟内存管理——大纲

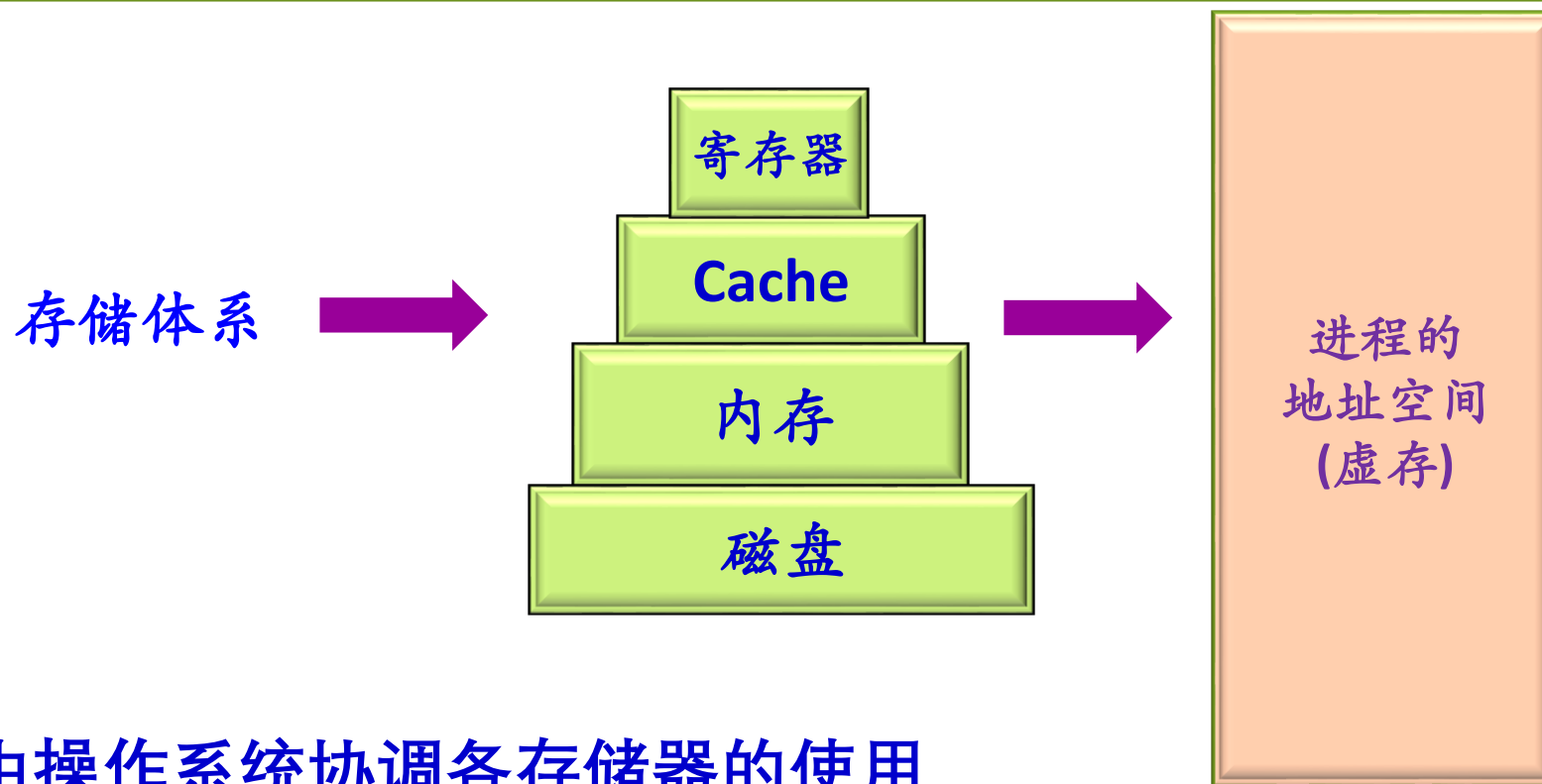
□ 虚拟内存管理

- 硬件机制（地址转换）、页表/页表项、页错误处理
- 各种软件策略
 - 读取策略、放置策略(placement)、置换策略(replacement)、驻留集策略、清除策略、装载控制策略
- 内存映射文件机制
- Windows虚拟内存管理

虚拟页式存储管理

基本思想、设计与实现问题、……

存储体系



- ▶ 由操作系统协调各存储器的使用
- ▶ 目的

“内存”速度尽量快，与CPU取指速度相匹配；“内存”容量尽量大，能装下当前运行的程序与数据

相关术语辨识

- ▶ 虚拟内存
- ▶ 虚拟地址空间
- ▶ 虚拟地址
- ▶ 虚拟存储技术

* 把物理内存与磁盘结合起来使用，得到一个容量很大的“内存”，即虚存
* 程序引用内存所使用的地址与内存物理地址是不同的，可被自动转换成物理地址
* 虚存大小受计算机系统寻址机制和可用磁盘容量的限制

分配给进程的虚拟内存

虚拟内存中某一位置的地址，该位置可以被访问，仿佛它是内存的一部分

当进程运行时，先将其一部分装入内存，另一部分暂时保存在磁盘；当要执行的指令或访问的数据不在内存时，由操作系统自动完成将它们从磁盘调入内存的工作

虚拟页式存储管理

Page Fault

► 基本思想

- 装载程序时，不是装入全部页面，而是装入几个甚至零个页面
- 如果进程执行时需要的页面不在内存，则动态装入所需页面
- 需要时，将内存中暂时不用的一些页面交换到磁盘，以便获得更多的内存空间

► 通常有两种方式

- 请求调页 (demand paging)
- 预先调页 (prepaging)

操作系统中的资源转换技术
——以 CPU 时间和磁盘空间换取物理内存空间

调页机制

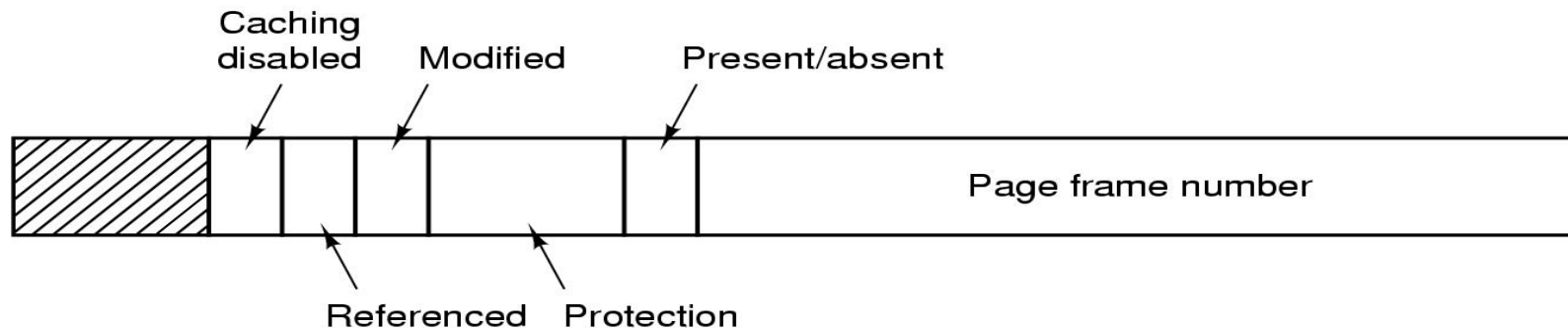
- ▶ 进程的虚拟地址空间与物理地址空间分离
- ▶ 调页系统的3个重要策略（Coffman & Denning）：
 - 1、系统何时把页面载入内存——取页策略（fetch policy）
 - 2、系统把页面放在何处——放置策略（placement policy）
 - 3、执行放置操作时发生页框不足时，如何选择从内存里“删除”的其他页框——置换策略（replacement policy）

设计与实现时要解决的问题

- ▶ 页表表项的设计
- ▶ 如何处理页表巨大的问题？
- ▶ 地址重定位与快表(TLB)
- ▶ 一种最常见的Page Fault → 缺页异常
- ▶ 驻留集管理
- ▶ 置换策略
- ▶ 清除策略
- ▶ 加载控制

1.页表表项设计

- ▶ 页框号、有效位、访问位、修改位、保护位等
 - ✓ 页框号（内存块号、物理页面号、页帧号）
 - ✓ 有效位（驻留位、中断位）：表示该页是在内存还是在磁盘（Valid、Present）
 - ✓ 访问位：引用位（Referenced、Accessed）
 - ✓ 修改位：查看此页是否在内存中被修改过（Dirty、Modified）
 - ✓ 保护位：读/写/执行（Protection）



i386页目录项和页表项

页目录项 PDE (Page Directory Entry)

PFN	Avail	G	PS	0	A	PCD	PWT	U/S	R/W	P
-----	-------	---	----	---	---	-----	-----	-----	-----	---

页表项 PTE (Page Table Entry)

PFN	Avail	G	0	D	A	PCD	PWT	U/S	R/W	P
-----	-------	---	---	---	---	-----	-----	-----	-----	---

PFN(Page Frame Number): 页框号

P(Present): 有效位

A(Accessed): 访问位

D(Dirty): 修改位

R/W(Read/Write): 只读/可读写

U/S(User/Supervisor): 用户/内核

PWT(Page Write Through): 缓存写策略

PCD(Page Cache Disable): 禁止缓存

PS(Page Size): 大页4M

2.多级页表

64位虚拟地址空间：
一张页表有
33554432 GB

► 计算一下

32位虚拟地址空间的页表规模？

页面大小为 4K；页表项大小为 4 字节；

用户地址空间为 $2G = 2^{31}$

则：一个进程有 2^{19} 页；其页表需要占 512 页

► 一个进程的页表的各页在内存中若不连续存放，则需要引入地址索引 → **页目录（一级页表）**

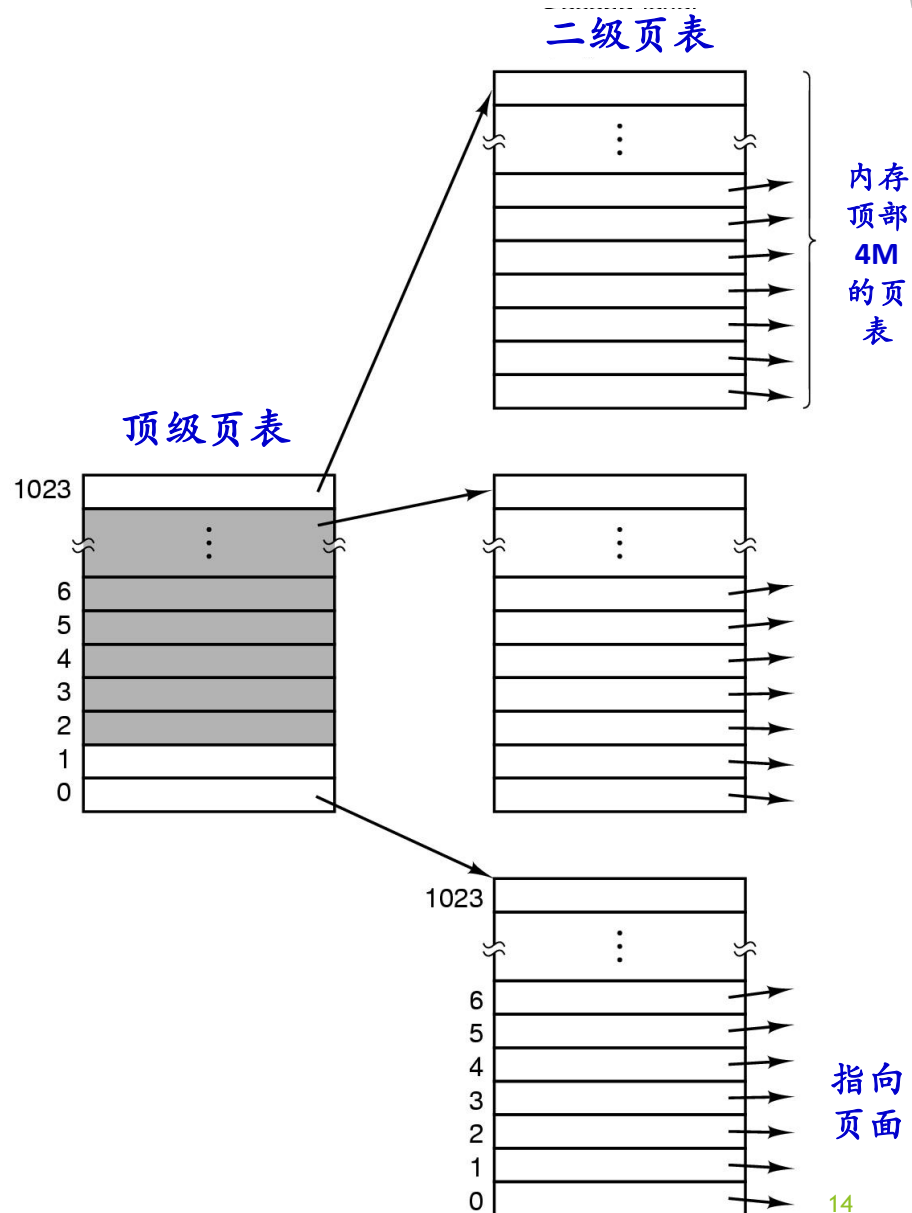
► 注意：页表本身也映射到虚存（进程运行时，部分页表映射到内存）

两级页表设计思想

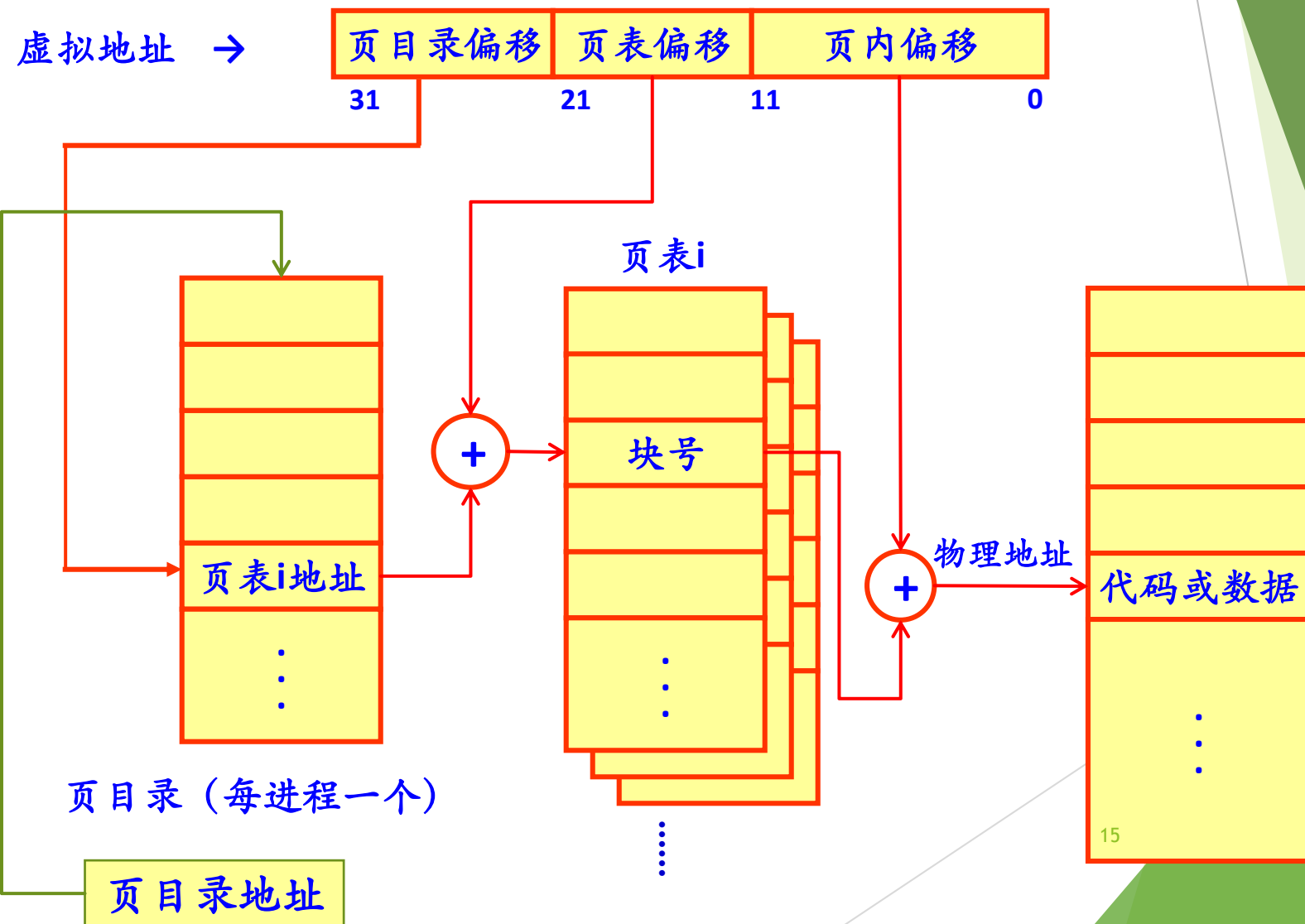
Bits	10	10	12
	PT1	PT2	Offset

(a)

相当于数组
转变为树形
结构

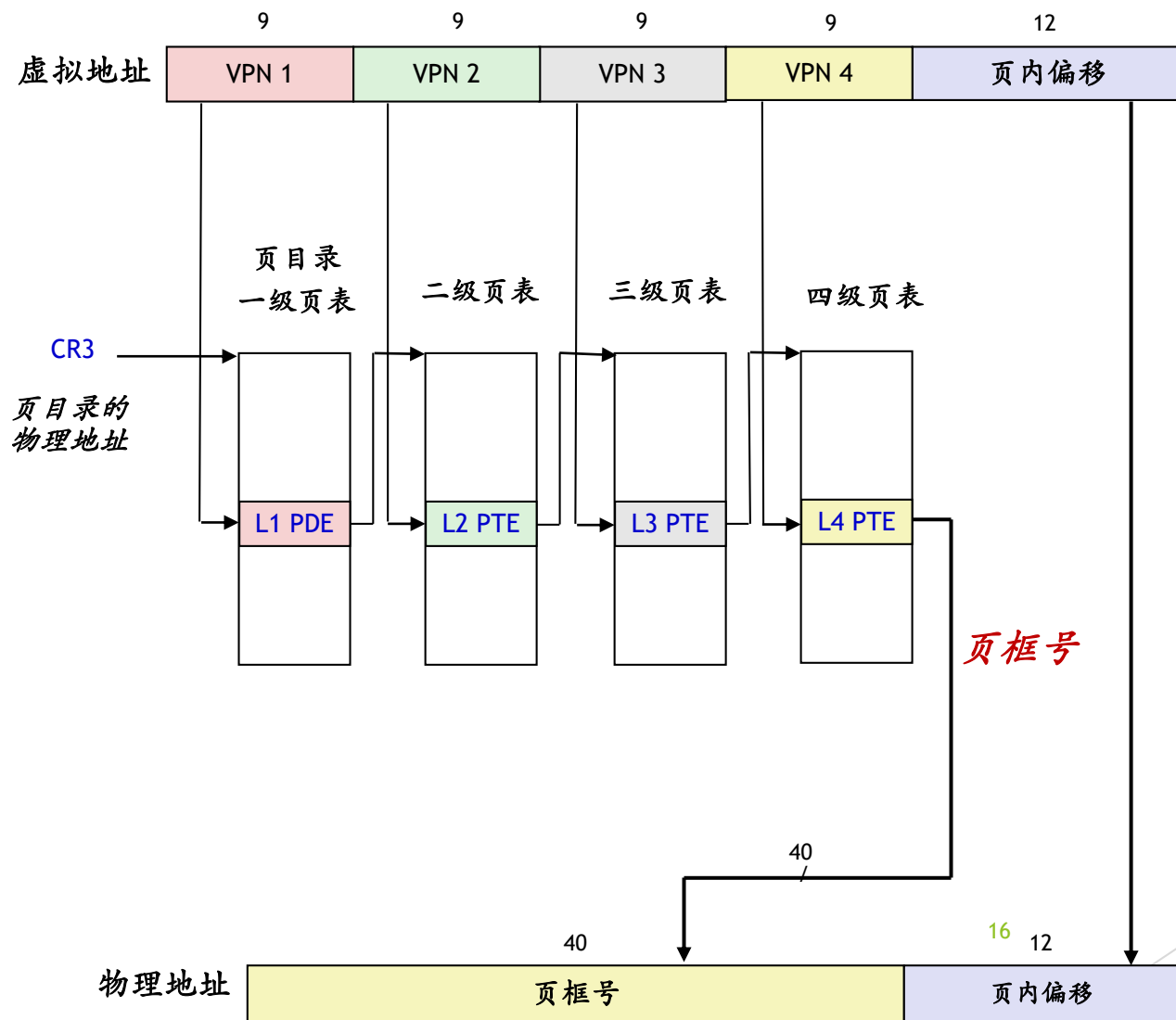


二级页表结构及地址映射

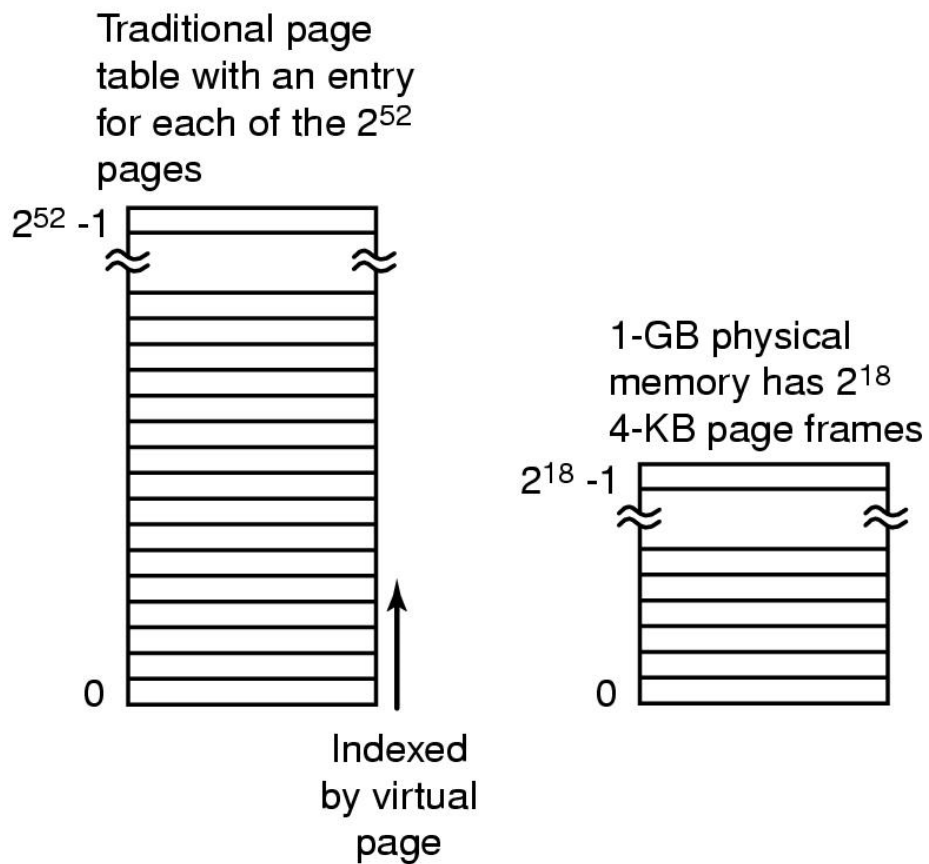


Core i7 页表结构

虚拟地址
空间 2^{48}



3.引入反转(反置、反向)页表



● 地址转换

-- 从虚拟地址空间出发：虚拟地址 $\rightarrow$ 查页表 $\rightarrow$ 得到页框号 $\rightarrow$ 形成物理地址

-- 每个进程一张页表

● 反转页表思路

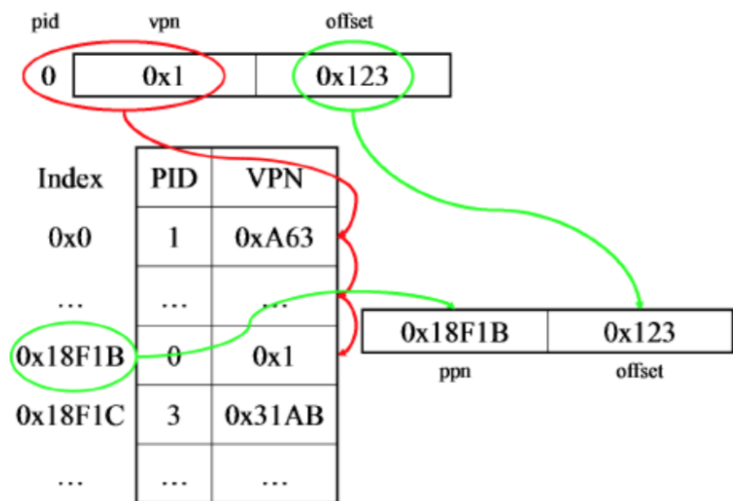
-- 从物理地址空间出发，系统建立一张页表

-- 页表项记录进程*i*的某虚拟地址(虚页号)与页框号的映射关系

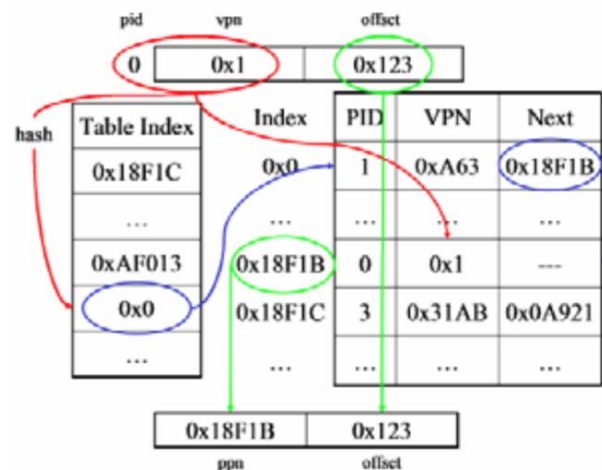
反转(反置、反向)页表示例

► 参考virginia的cs333

Accessing Inverted Page Table



Accessing Hashed Inverted Page Table



小结

- ▶ PowerPC、UltraSPARC和IA-64 等体系结构采用

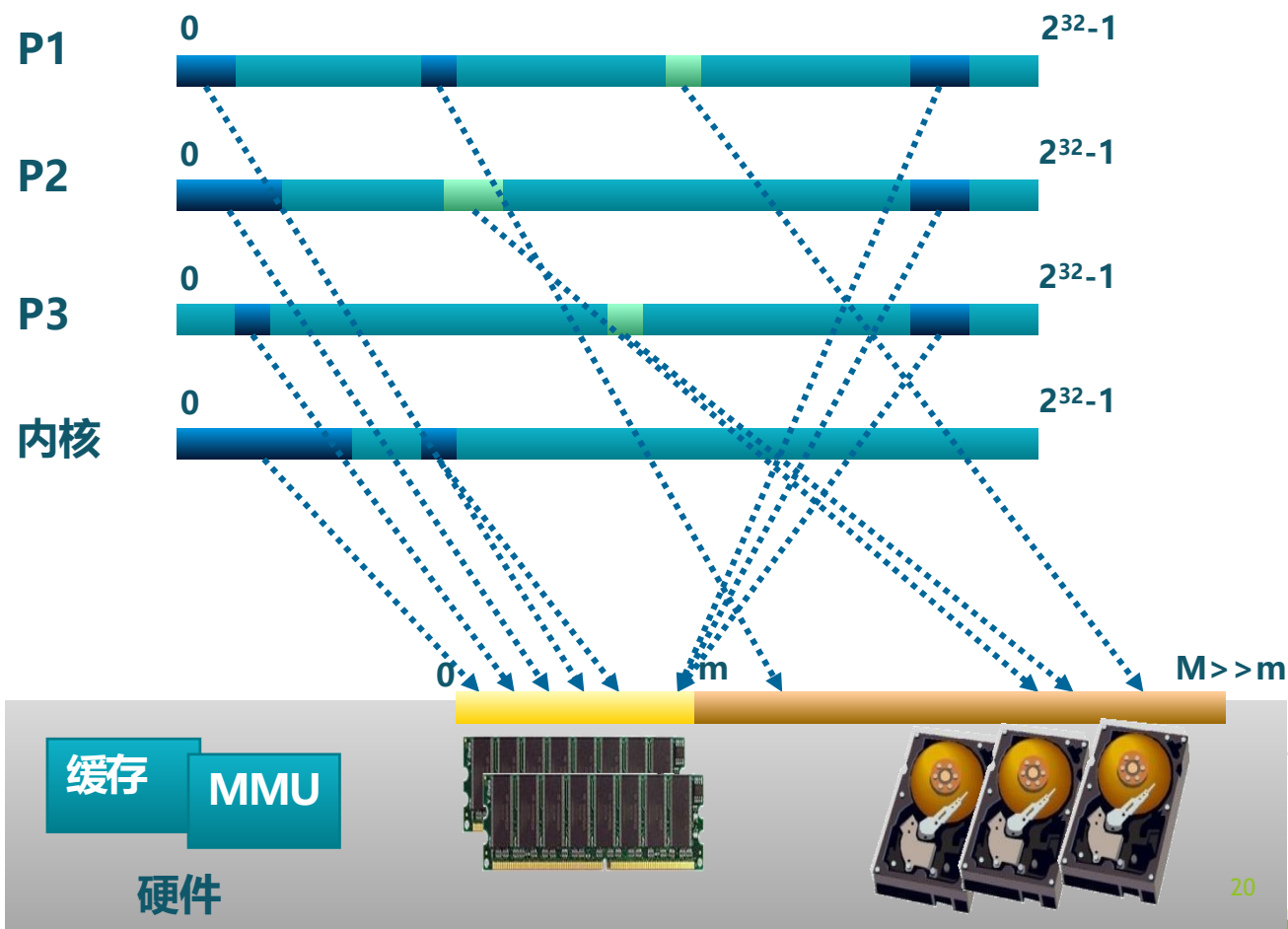
做法：

- ▶ 将虚拟地址的页号部分与进程pid映射到一个散列值
- ▶ 散列值指向一个反转页表
- ▶ 需要拉链解决冲突问题

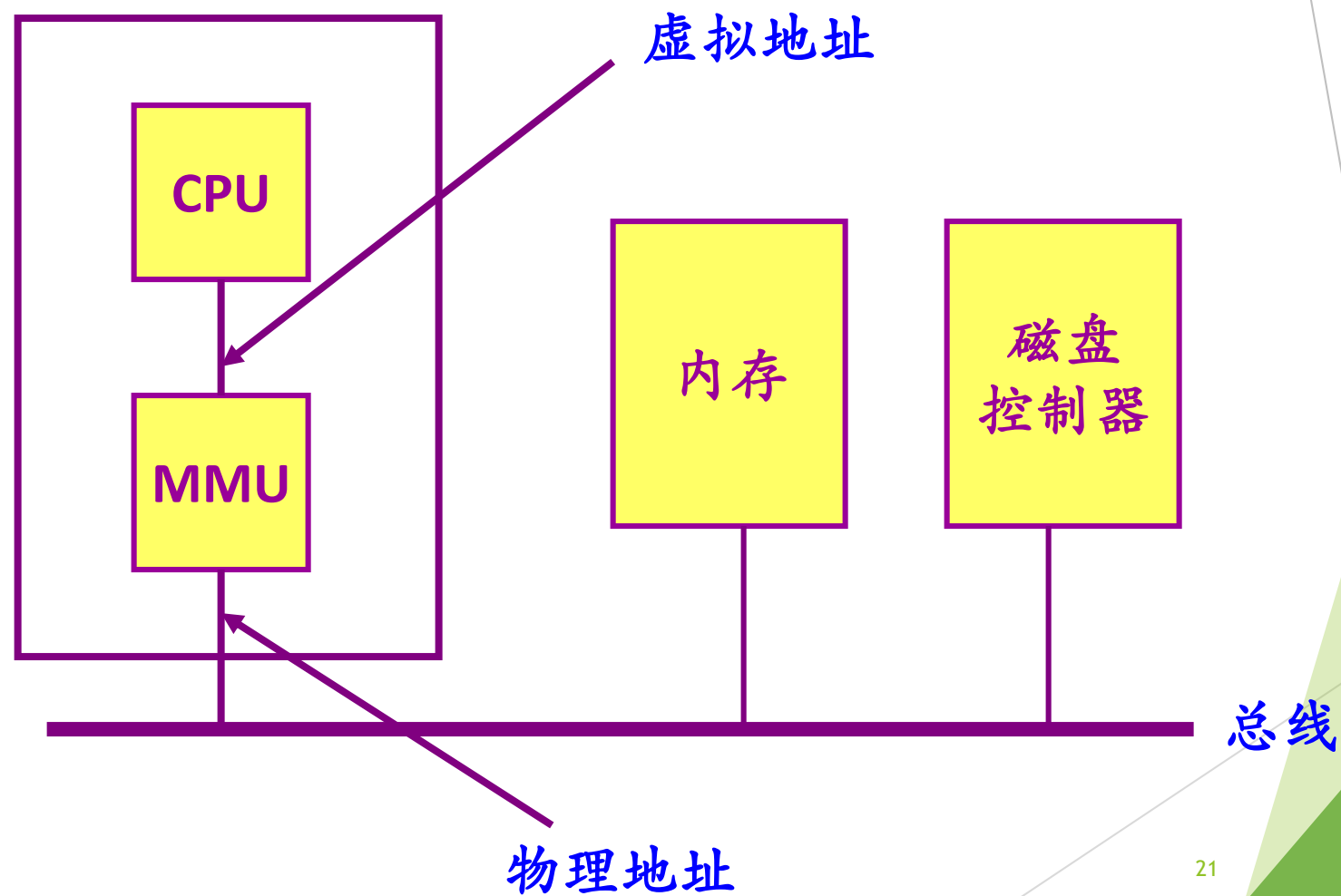
- ▶ 反转页表大小与实际内存成固定比例，与进程个数无关

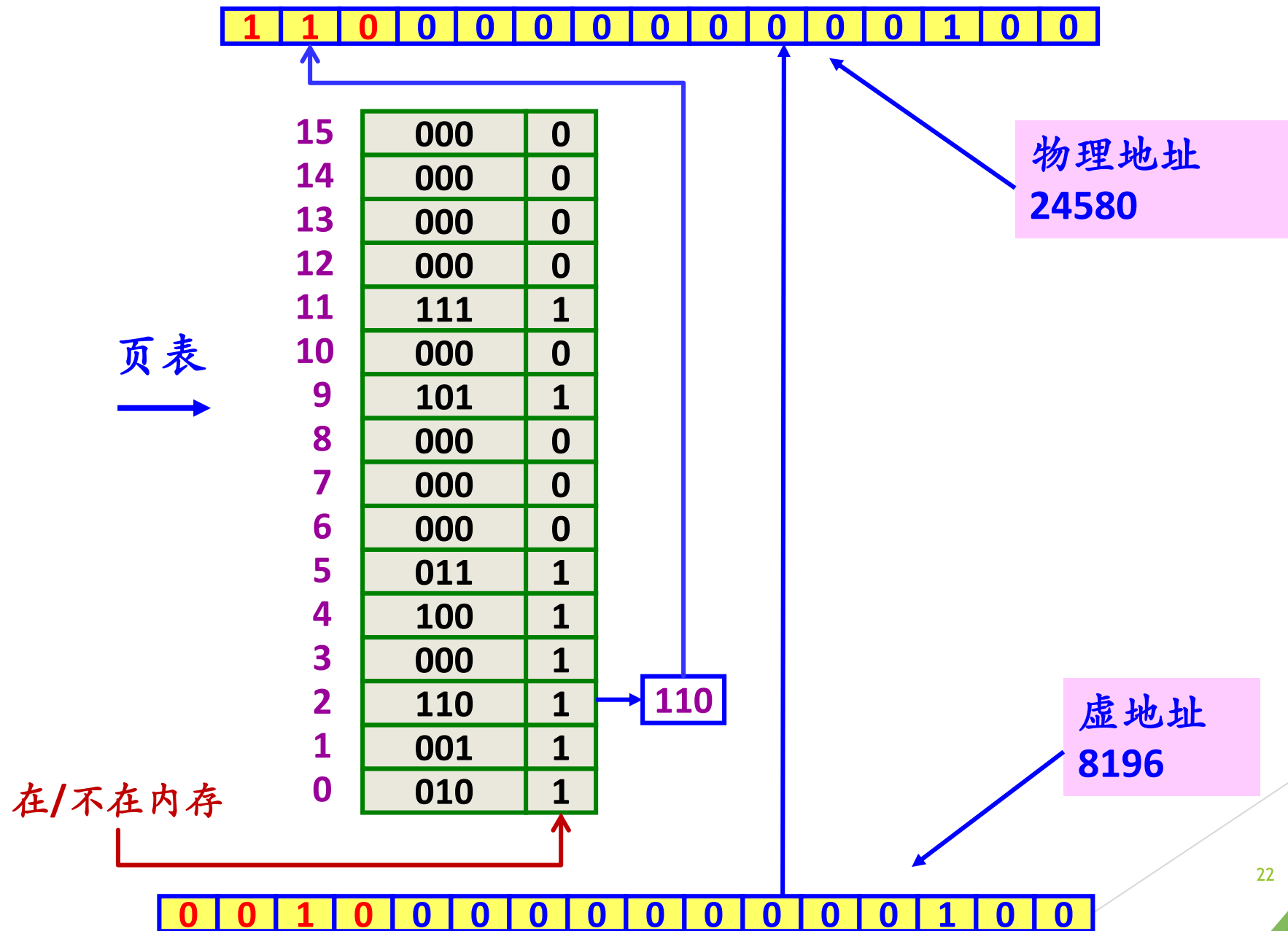
操作系统的存储抽象

► 操作系统对存储的抽象：地址空间



MMU:内存管理单元





4.地址转换

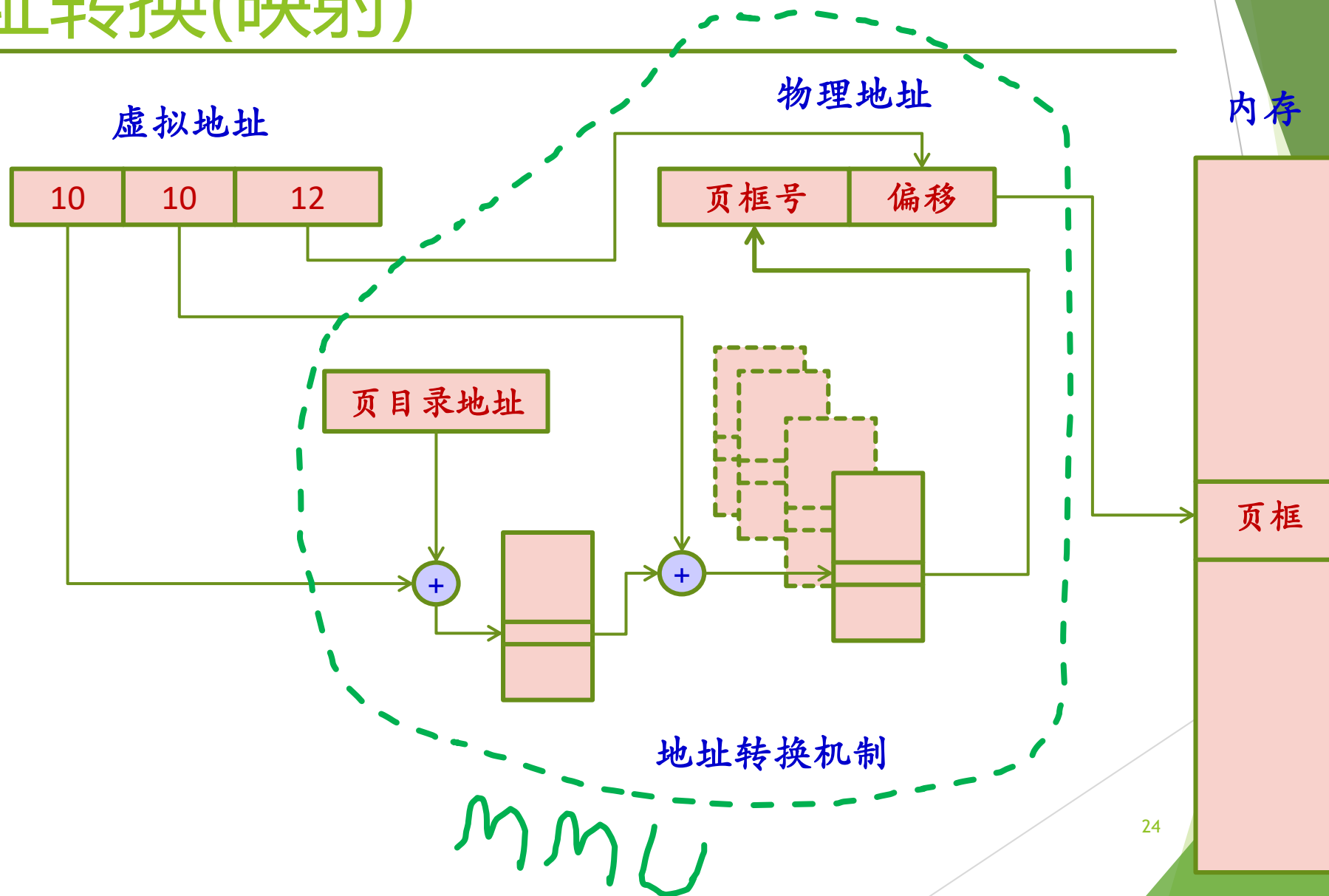
► 地址转换过程(硬件机制)

if (虚拟页面不在内存、页面非法、或者被保护) {
 硬件产生异常，陷入操作系统，执行页面错误
 服务程序 (Page Fault)
}
else {
 页框号 = 页表[虚页号]
 物理地址 = 页框号 拼接 页内偏移
}



内核虚拟内存
用户栈 (运行时创建的)
共享库的存储器 映射区域
运行时堆 (在运行时由malloc 创建的)
读/写数据
只读的代码和数据

地址转换(映射)



5.快表(TLB)的引入

问题

- 页表 → 两次或两次以上的内存访问
- CPU的指令处理速度与内存指令的访问速度差异大，CPU的速度得不到充分利用

如何加快地址映射速度，以改善系统性能？

程序访问的局部性原理 → 引入快表

快表是什么？

- 相联存储器 (associative memory)

特点：按内容并行查找

- TLB —— Translation Look-aside Buffers
- 保存正在运行进程的页表的子集(部分表项)

- 快表的作用：加快地址映射速度，改善系统性能
- 工作原理：采用联想映射技术按内容同时查找

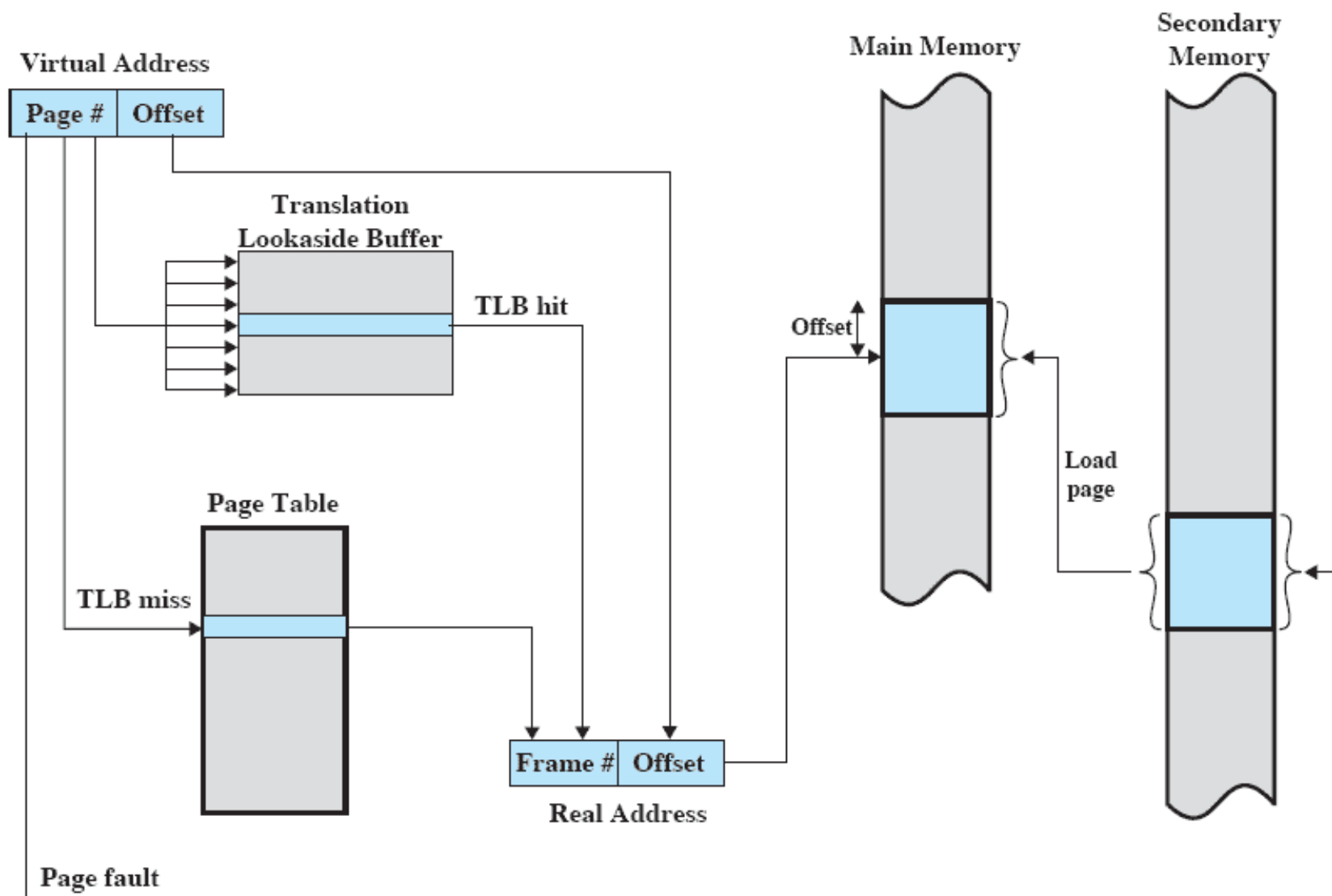
快表TLB的刷新问题：

- 进程切换，TLB刷新，TLB未命中，性能下降
- PCID (Process Context Identifier) / ASID (Address Space Identifier) 功能

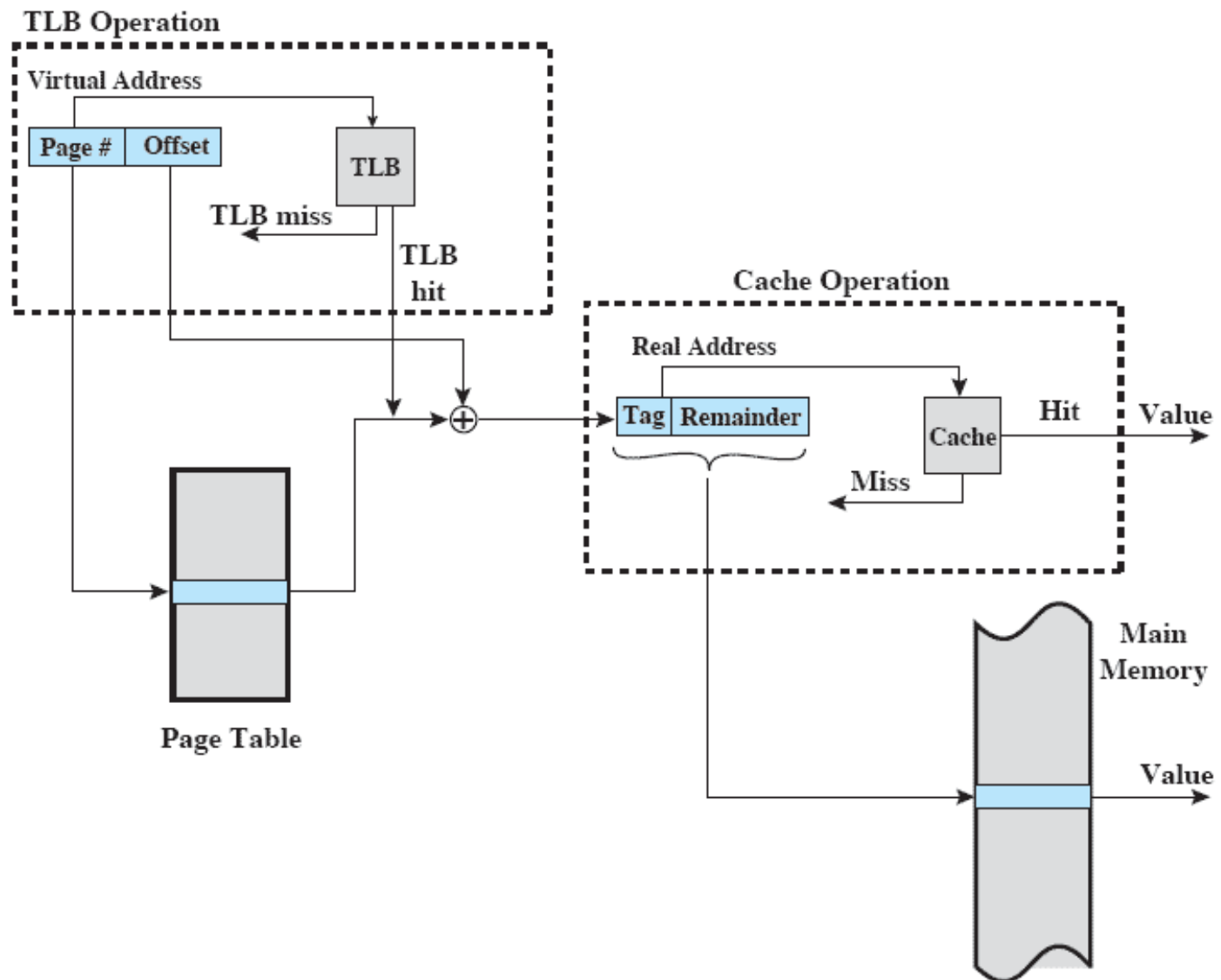
快表的
大小、位置

快表的
置换
问题？

TLB的使用



TLB与高速缓存操作



6.缺页异常处理

预取一些
页面

► 一种Page Fault

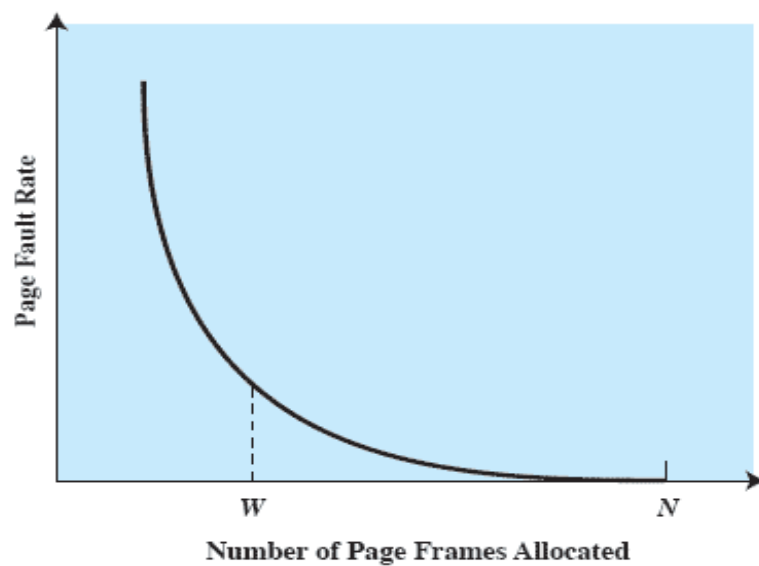
页错误、页面错误、页故障、页失效

- 在地址映射过程中，硬件检查页表时发现所要访问的页不在内存，则产生该异常——缺页异常
- 操作系统执行缺页异常处理程序：获得磁盘地址，启动磁盘，将该页调入内存。
 - 如果内存中有空闲页框，则分配一页，将新调入页装入内存，并修改页表中相应页表项的驻留位及相应的页框号
 - 若内存中没有空闲页框，则要置换某一页；若该页在内存期间被修改过，则要将其写回磁盘

7.驻留集管理

▶ 驻留集大小

给每个进程分配多少页框？



● 固定分配策略

进程创建时确定

可以根据进程类型（交互、批处理、应用类）或者基于程序员或系统管理员的需要来确定

● 可变分配策略

根据缺页率评估进程局部性表现

- ✓ 缺页率高→增加页框数
- ✓ 缺页率低→减少页框数
- ✓ 系统开销

页面置换算法

基本思想、设计与实现问题、……

1. 置换问题

► 置换范围

计划置换页面的集合是局限在产生缺页异常的进程，还是所有进程的页框？

► 局部置换策略

仅在产生本次缺页的进程的驻留集中选择

► 全局置换策略

将内存中所有未锁定的页框都作为置换的候选

	局部置换	全局置换
固定分配	√	--
可变分配	√	√

- 1、当一个新进程装入内存时，给它分配一定数目的页框，使用预先分页或请求分页填满这些页框
2. 当发生缺页异常时，从该进程的驻留集中选择一页用于置换
3. 不断重新评估进程的页框分配情况，增加或减少分配给它的页框，以提高整体性能

置换策略

放置 (placement, 分配)
置换 (replacement, 替换、淘汰)

- 决定置换当前内存中的哪一个页框
- 所有策略的目标 → **置换最近最不可能访问的页**
- 根据局部性原理，最近的访问历史和最近将要访问的模式间存在相关性，因此，大多数策略都**基于过去的行为来预测将来的行为**
- 注意：置换策略设计得越精致、越复杂，实现的软硬件开销就越大
- 约束条件：不能置换被锁定的页框

典型设计思路

页框锁定

特别是正在I/O的
内存页面

为什么要锁定页面？

- ▶ 采用虚存后：
 开销 → 使程序运行时间变得不确定
- ▶ 给每一页框增加一个锁定位
- ▶ 通过设置相应的锁定位，不让操作系统将进程使用的页面换出内存，避免产生由交换过程带来的不确定的延迟
- ▶ 例如：**操作系统核心代码、关键数据结构、I/O缓冲区**

Windows中的VirtualLock和VirtualUnlock函数

2.页面置换(replacement)算法

又称页面淘汰（替换）算法

最优算法→最近未使用→先进先出→第二次机会→时钟算法→
最近最少使用→最不经常使用→老化算法→工作集→工作集时
钟

理想 (最佳、最优)置换算法 (OPT)

- ▶ 设计思想:

置换以后不再需要的或最远的将来才会用到的页面

- ▶ 实现?

- ▶ 作用?

先进先出页面置换算法 (FIFO)

- 选择在内存中驻留时间最长的页并置换它

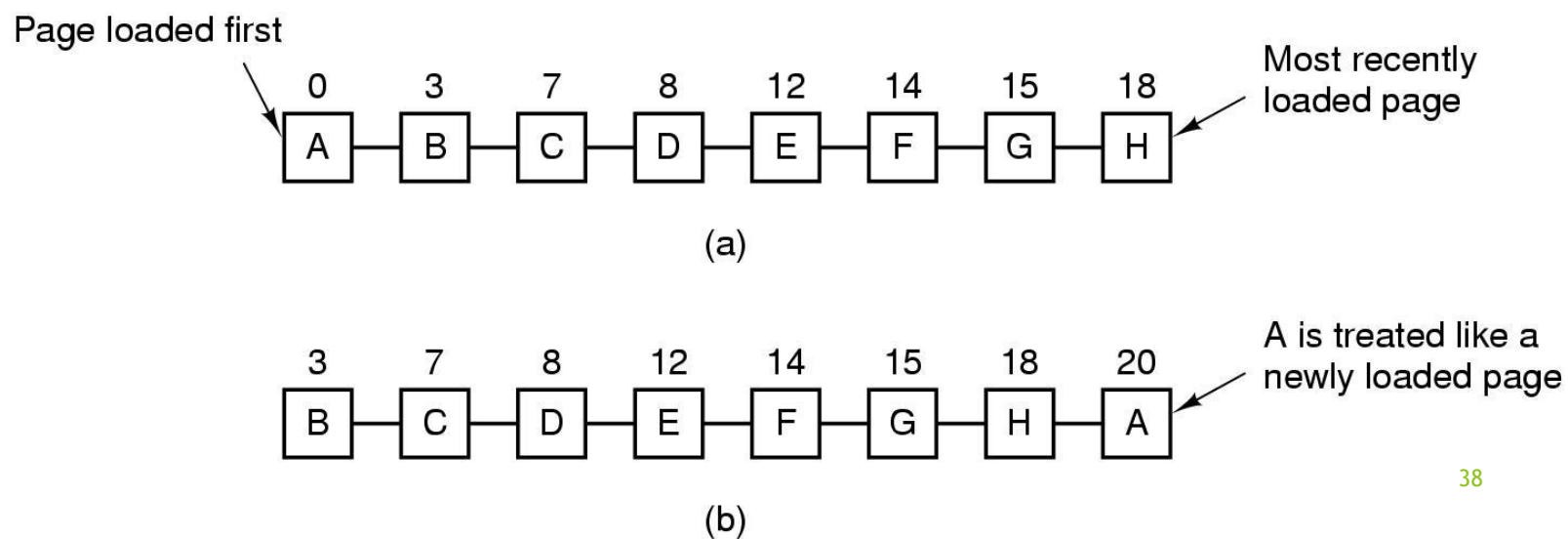
对照：超市撤换商品

- 实现：页面链表法

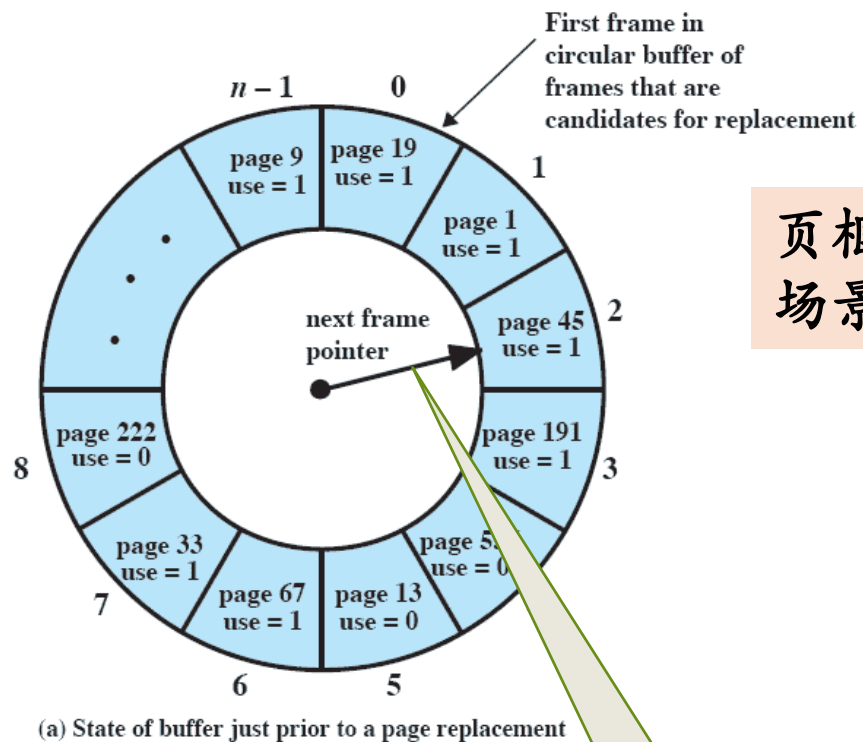
第二次机会置换算法(SCR)

SCR-Second Chance

按照先进先出算法选择某一页面，检查其访问位R，如果为0，则置换该页；如果为1，则给第二次机会，并将访问位置0

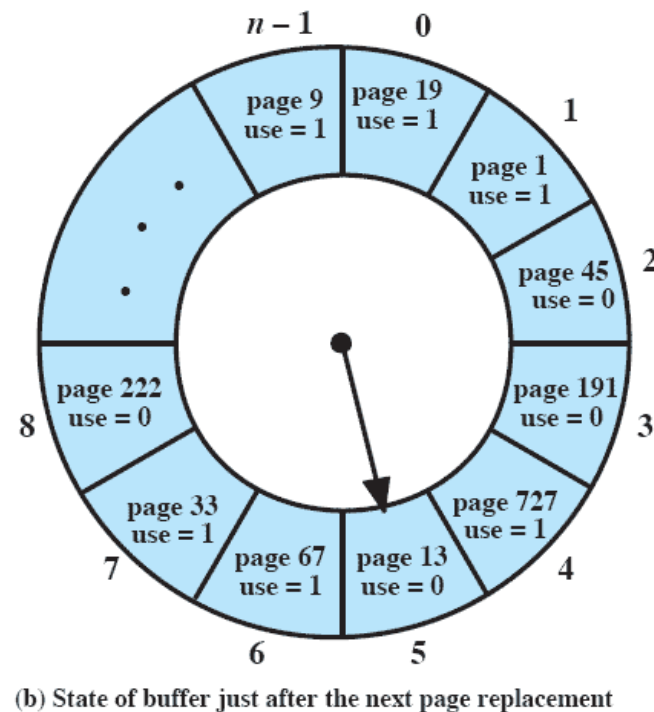


时钟算法(Clock)



页框组织成循环缓冲区
场景：页727进入

下一页
框指针



最近未使用算法(NRU)(1/2)

Not Recently Used

选择在最近一段时间内未使用过的一页并置换

实现：根据页表表项的两位
访问位（R），修改位（M）

如果硬件没有这些位，
则可用软件模拟（做
标记）

启动一个进程时，R、M位置0
R位被定期清零（复位）

最近未使用算法(NRU)(2/2)

发生缺页异常时，操作系统检查R，M：

第0类：无访问，无修改

第1类：无访问，有修改

第2类：有访问，无修改

第3类：有访问，有修改

算法思想：

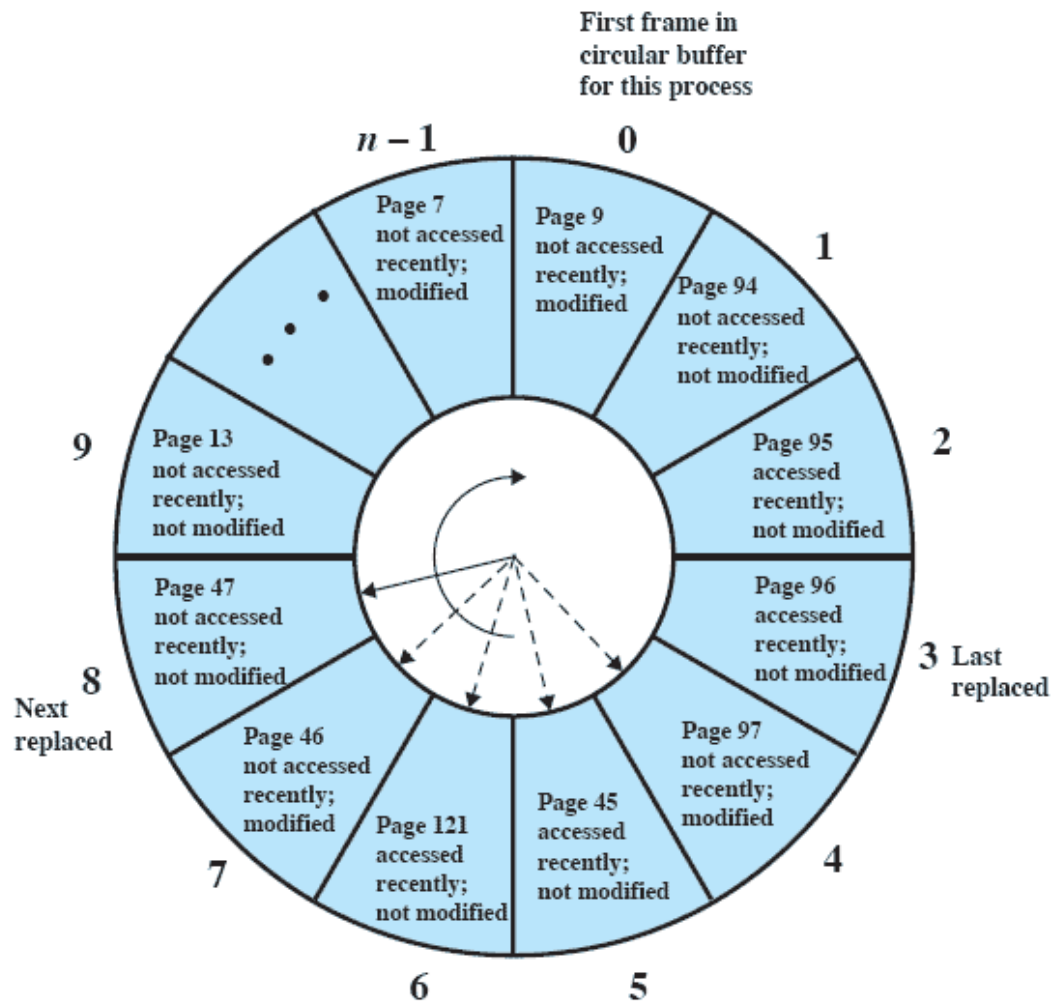
随机从编号最小的非空类中选择一页置换

NRU的时钟算法实现

1. 从指针的当前位置开始，扫描页框缓冲区，选择遇到的第一个页框 ($r=0$; $m=0$) 用于置换(本扫描过程中，对访问位不做任何修改)
2. 如果第1步失败，则重新扫描，选择第一个 ($r=0$; $m=1$) 的页框 (本次扫描过程中，对每个跳过的页框，将其访问位设置成0)
3. 如果第2步失败，指针将回到它的最初位置，并且集合中所有页框的访问位均为0。重复第1步，并且，如果有必要，重复第2步。这样将可以找到供置换的页框

提示：与上面介绍的NRU有区别

时钟算法实现



该策略用于较早版本的
macintosh虚拟存储方案

优先选择不需
要写回磁盘的
页面，节省时间

最近最久未使用算法(LRU)

Least Recently Used

选择最后一次访问时间距离当前时间最长的一页并置换
即置换未使用时间最长的一页

- 性能接近**OPT**
- 实现：时间戳 或 维护一个访问页的栈
→ 开销大

LRU算法的一种硬件实现

思考一下：
解释其理由

► 页面访问顺序0, 1, 2, 3, 2, 1, 0, 3, 2, 3

Page				
	0	1	2	3
0	0	1	1	1
1	0	0	0	0
2	0	0	0	0
3	0	0	0	0

(a)

Page				
	0	1	2	3
0	0	0	1	1
1	1	0	1	1
2	0	0	0	0
3	0	0	0	0

(b)

Page				
	0	1	2	3
0	0	0	0	1
1	1	0	0	1
2	1	1	0	1
3	0	0	0	0

(c)

Page				
	0	1	2	3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	0
3	1	1	1	0

(d)

Page				
	0	1	2	3
0	0	0	0	0
1	1	0	0	0
2	1	1	0	1
3	1	1	0	0

(e)

0	0	0	0
1	0	1	1
1	0	0	1
1	0	0	0

(f)

0	1	1	1
0	0	1	1
0	0	0	1
0	0	0	0

(g)

0	1	1	0
0	0	1	0
0	0	0	0
1	1	1	0

(h)

0	1	0	0
0	0	0	0
1	1	0	1
1	1	0	0

(i)

0	1	0	0
0	0	0	0
1	1	0	0
1	1	1	0

(j)

最不经常使用算法(NFU)

Not Frequently Used

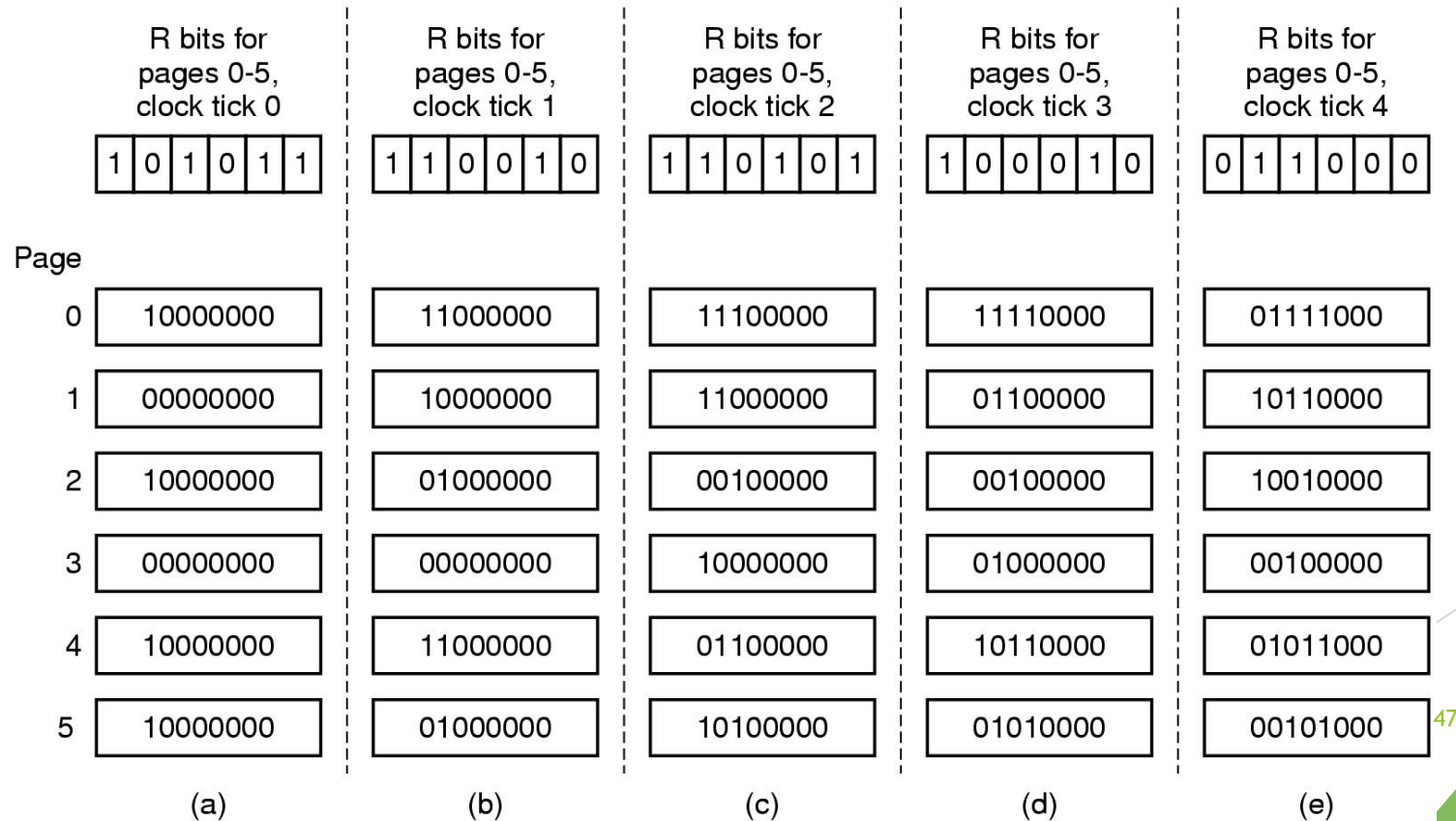
选择访问次数最少的页面置换

- LRU的一种软件解决方案
- 实现：
 - 软件计数器，一页一个，初值为0
 - 每次时钟中断时，计数器加R
 - 发生缺页异常时，选择计数器值最小的一页置换

老化算法(Aging)

思考：与LRU的比较

改进（模拟LRU）：计数器在加R前先右移一位
R位加到计数器的最左端



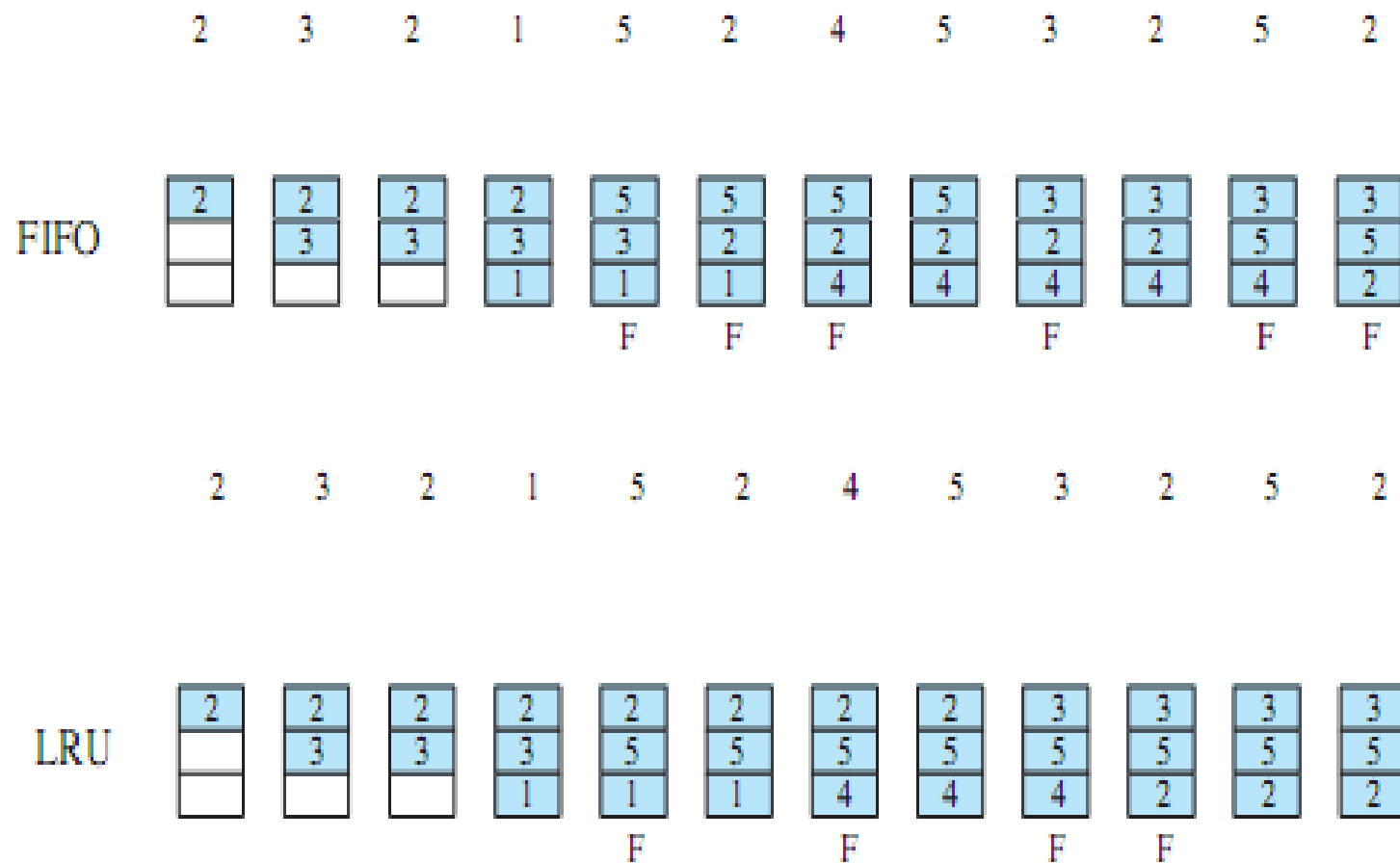
3.页面置换算法应用

例子：系统给某进程分配3个页框(固定分配策略)，初始为空
进程执行时，页面访问顺序为：

2 3 2 1 5 2 4 5 3 2 5 2

要求：计算应用FIFO、LRU、OPT算法时的缺页次数

应用FIFO、LRU页面置换算法



应用OPT页面置换算法

	2	3	2	1	5	2	4	5	3	2	5	2																																				
OPT	<table><tr><td>2</td></tr><tr><td></td></tr><tr><td></td></tr></table>	2			<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td></td></tr></table>	2	3		<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td></td></tr></table>	2	3		<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>1</td></tr></table>	2	3	1	<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>5</td></tr></table> F	2	3	5	<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>5</td></tr></table>	2	3	5	<table><tr><td>4</td></tr><tr><td>3</td></tr><tr><td>5</td></tr></table> F	4	3	5	<table><tr><td>4</td></tr><tr><td>3</td></tr><tr><td>5</td></tr></table>	4	3	5	<table><tr><td>4</td></tr><tr><td>3</td></tr><tr><td>5</td></tr></table>	4	3	5	<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>5</td></tr></table> F	2	3	5	<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>5</td></tr></table>	2	3	5	<table><tr><td>2</td></tr><tr><td>3</td></tr><tr><td>5</td></tr></table>	2	3	5
2																																																
2																																																
3																																																
2																																																
3																																																
2																																																
3																																																
1																																																
2																																																
3																																																
5																																																
2																																																
3																																																
5																																																
4																																																
3																																																
5																																																
4																																																
3																																																
5																																																
4																																																
3																																																
5																																																
2																																																
3																																																
5																																																
2																																																
3																																																
5																																																
2																																																
3																																																
5																																																

4. Belady现象

例子：系统给某进程分配 m 个页框，初始为空
页面访问顺序为

1 2 3 4 1 2 5 1 2 3 4 5

采用FIFO算法，计算当 $m=3$ 和 $m=4$ 时的缺页异常次数

$m=3$ 时，缺页异常9次； $m=4$ 时，缺页异常10次

注：FIFO页面置换算法会产生异常现象（Belady现象），
即：当分配给进程的物理页面数增加时，缺页次数反而增加

5.影响缺页次数的因素

- 页面置换算法 *
- 页面本身的大小 ✓
- 程序的编制方法 ✓
- 分配给进程的物理页面数 ✓

颠簸 (Thrashing, 抖动)

虚存中，页面在内存与磁盘之间频繁调度，使得调度页面所需的时间比进程实际运行的时间还多，这样导致系统效率急剧下降，这种现象称为颠簸或抖动

页面尺寸问题

- ▶ 确定页面大小对于分页的硬件设计非常重要而对于操作系统是个可选的参数

- ▶ 要考虑的因素：

- ▶ 内部碎片
- ▶ 页表长度
- ▶ 辅存的物理特性

- ▶ Intel 80x86/Pentium：4096 或 4M

- ▶ 多种页面尺寸：为有效使用TLB带来灵活性，但给操作系统带来复杂性

小页面？
大页面？

最优页面大小
 $P = \sqrt{2se}$

程序编制方法对缺页次数的影响

例子：系统给某进程分配了一个页框；页面大小4K；

矩阵A[1024][1024]；按行存放

程序编制方法1:

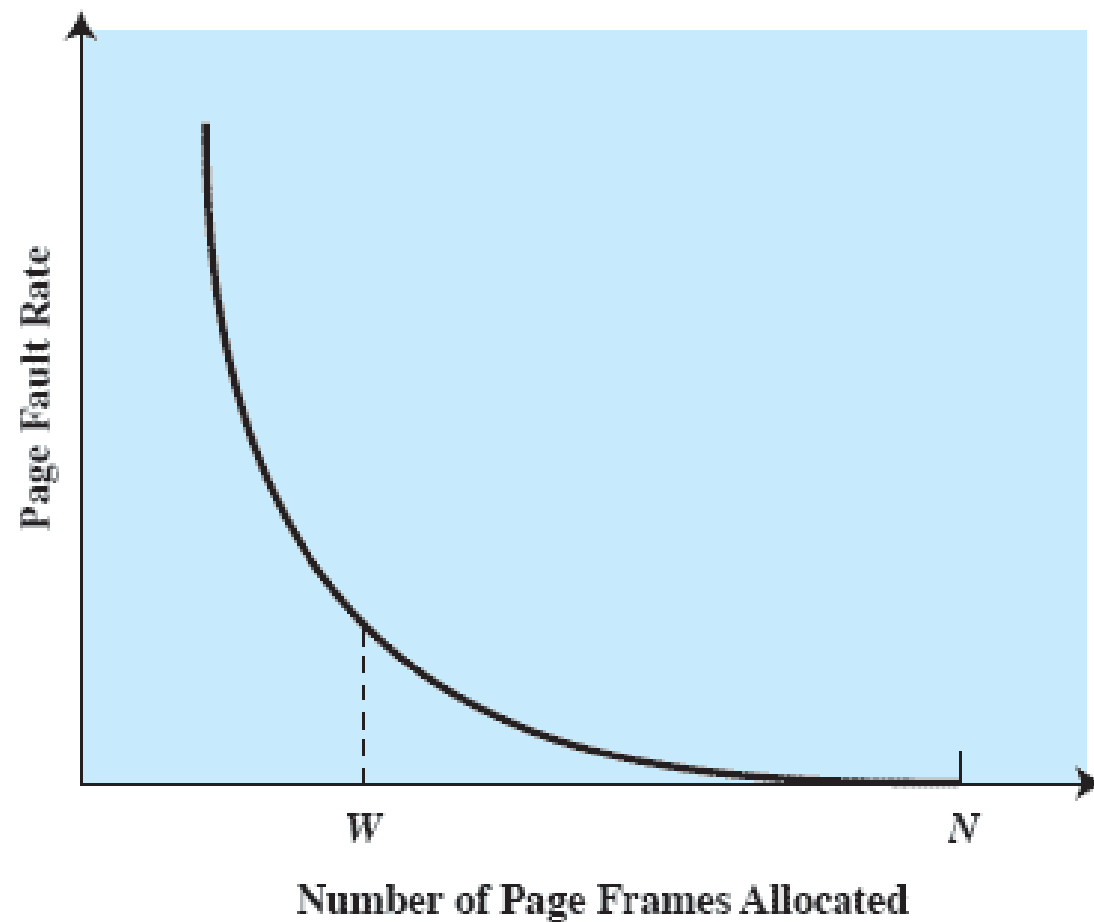
```
for (j = 0; j < 1024; j++)  
    for (i = 0; i < 1024; i++)  
        A[i][j] = 0;
```

程序编制方法2:

```
for (i=0; i<1024; i++)  
    for (j=0; j<1024; j++)  
        A[i][j] = 0;
```

缺页
几次?

分配给进程的页框数与缺页率的关系



6.工作集(Working Set)模型(1/3)

基本思想：

根据程序的局部性原理，一般情况下，进程在一段时间内总是集中访问一些页面，这些页面称为**活跃页面**，如果分配给一个进程的页框太少了，使该进程所需的活跃页面不能全部装入内存，则进程在运行过程中将频繁发生中断

如果能为进程提供与活跃页面数相等的页框数，则可减少缺页异常次数

由Denning提出(1968)

工作集(Working Set)模型(2/3)

工作集 $W(t, \Delta)$

= 该进程在过去的 Δ 个虚拟时间单位中使用的虚拟页面集合

工作集内容取决于三个因素：

- 访页序列特性
- 当前时刻 t
- 工作集窗口长度(Δ)

窗口越大，工作集就越大

工作集(Working Set)模型(3/3)

例子:

26157775162341234443434441327

|←→| t_1 |←→| t_2

$W(t_1, 10) = \{1, 2, 5, 6, 7\}$ $W(t_2, 10) = \{3, 4\}$

2 6 1 5 7 7 7 5 1 6 2 3 4 1 2 3 4 4 4 3 4 3 4 4 4 1 3 2 7

←→
 Δ t

$W(t, \Delta) = \{2, 2, 5, 5, 5, 5, 7\}$

工作集与驻留集

- 驻留集：当前时刻，进程实际驻留在内存当中的页框集合

两者的关系：

- ✓ 工作集是进程在运行过程中固有的性质
- ✓ 驻留集取决于系统分配给进程的页框数和页面置换算法
- ◆ 监视每个进程的工作集
- ◆ 周期性地从一个进程的驻留集中移去那些不在它的工作集中的页（可使用LRU策略）
- ◆ 只有当一个进程的工作集在内存中时，才可以更好地执行该进程，即进程的驻留集包括了它的工作集）

工作集算法(1/3)

基本思路：

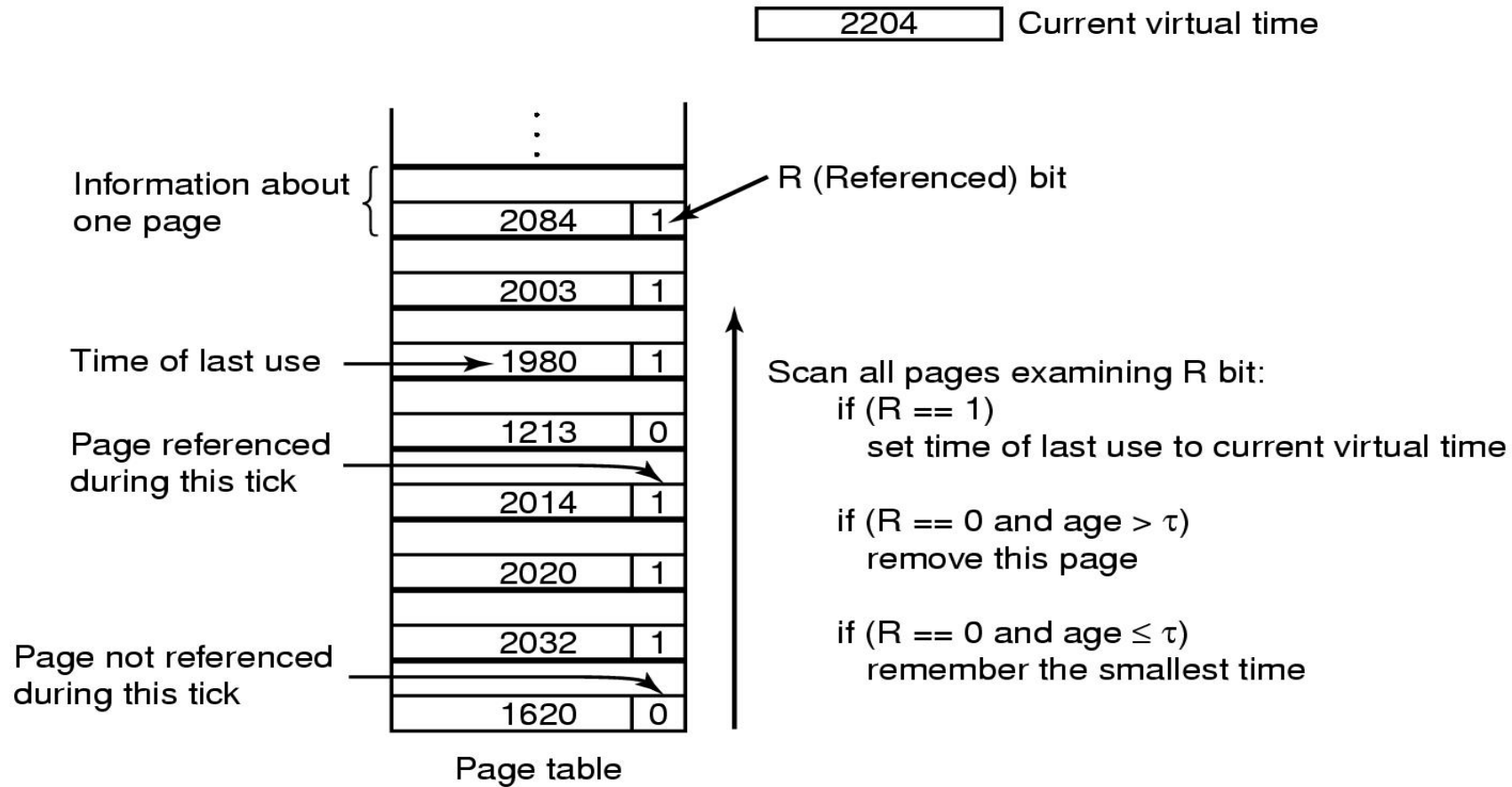
找出一个不在工作集中的页面并置换它

思路：

- ▶ 每个页表项中有一个字段：记录该页面最后一次被访问的时间
- ▶ 设置一个时间值 T
- ▶ 判断：

根据一个页面的访问时间是否落在“当前时间- T ”之前或之中决定其在工作集之外还是之内

工作集算法(2/3)



工作集算法(3/3)

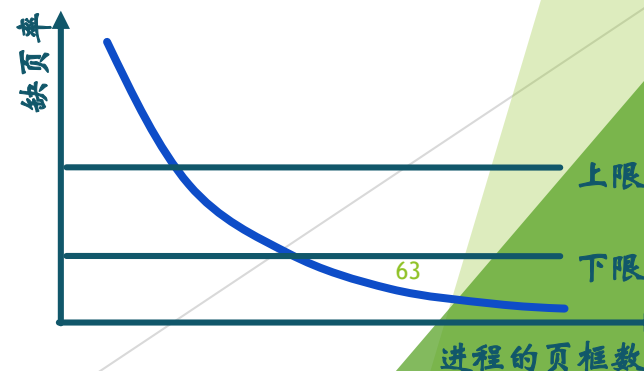
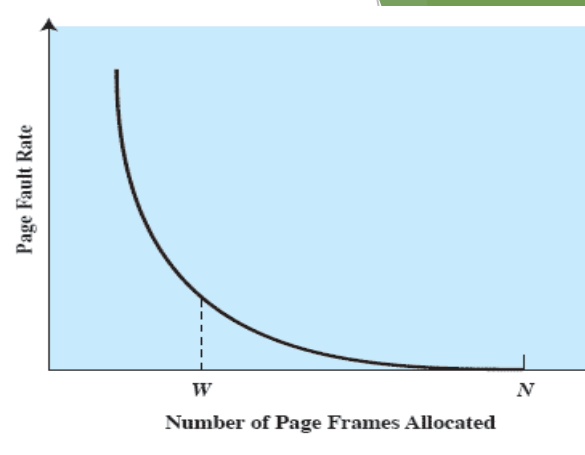
实现：

扫描所有页表项，执行操作

1. 如果一个页面的R位是1，则将该页面的最后一次访问时间设为当前时间，将R位清零
2. 如果一个页面的R位是0，则检查该页面的访问时间是否在“**当前时间-T**”之前
 - (1) 如果是，则该页面为被置换的页面；
 - (2) 如果不是，记录当前所有被扫描过页面的最后访问时间里面的最小值。扫描下一个页面并重复1、2

工作集算法讨论

- ▶ 工作集策略的思想是有效的
- ▶ 许多操作系统都试图采用近似工作集策略
- ▶ 其中的一种方法是考虑进程的缺页率，通过监视缺页率达到类似的结果——缺页率算法
 - 当增大一个进程的驻留集时，缺页率会下降，工作集的大小会降到图中W点所标记的位置
 - 方法：设置一个进程的缺页率的最小阈值和最大阈值，根据此增减驻留集大小



页面置换算法小结

算法	评价
OPT	不可实现，但可作为基准
NRU	LRU的很粗略的近似
FIFO	可能淘汰重要的页面
Second Chance	比FIFO有很大的改善
Clock	现实的
LRU	很优秀，但很难实现
NFU	LRU的相对粗略的近似
Aging	非常近似LRU的有效算法
Working set	实现起来开销很大
WSClock	好的有效的算法

7.清除策略(1/2)

- 分页系统工作的**最佳状态**：发生缺页异常时，系统中有大量的空闲页框
- **结论：保存一定数目的页框供给比使用所有内存并在需要时搜索一个页框有更好的性能**
- 设计一个分页守护进程（paging daemon），多数时间处于睡眠状态，可定期唤醒以检查内存状态
- 如果空闲页框过少，分页守护进程通过预设的页面置换算法选择页面换出内存
- 如果页面装入内存后被修改过，则将它们写回磁盘
分页守护进程可保证所有的空闲页框是“干净”的

清除策略(2/2)

- 当需要使用一个已置换出的页框时，如果该页框还没有被新内容覆盖，则将它从空闲页框缓冲池中移出即可恢复该页面

清除策略实现(A.S.Tanenbaum)

- 使用一个双指针时钟，前指针由分页守护进程控制：当它指向一个“脏”页面时，就把该页面写回磁盘，前指针向前移动；当它指向一个干净页面时，仅仅向前移动指针
- ✓ 后指针用于页面置换，与标准时钟算法一样
- ✓ 由于分页守护进程的工作，后指针命中干净页面的概率会增加

页缓冲技术

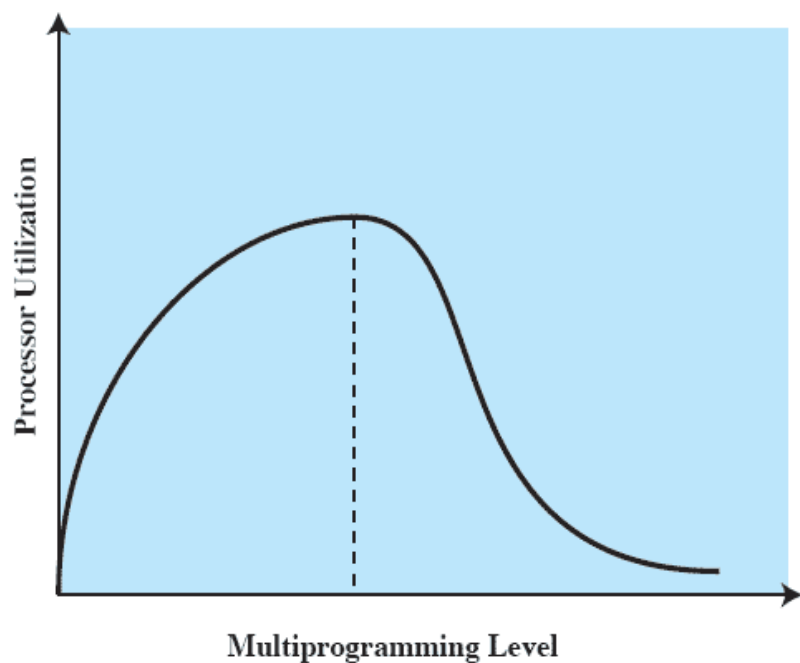
► 目的：提高性能

► 思路

- 不丢弃置换出的页，将它们放入两个表之一：如果未被修改，则放到空闲页链表中，如果修改了，则放到修改页链表中
- 被修改的页以簇方式写回磁盘(不是一次只写一个，减少I/O操作的数量，从而减少了磁盘访问时间)
- 被置换的页仍然保留在内存中，一旦进程又要访问该页，可以迅速将它加入该进程的驻留集合(代价很小)

8.加载控制

系统并发度：驻留在内存中的进程数目
通过调节并发进程数进行系统负载控制



解决方案：进程挂起
即释放一部分进程所占有的页面
→将它们交换到磁盘上

问题：
选择哪些进程？

其他设计及实现问题

内存映射文件……

内存映射文件

► 基本思想

进程通过一个系统调用（例如mmap）将一个文件映射到其虚拟地址空间的一个区域，访问这个文件就象访问内存中的一个大数组，而不是对文件进行读写

- 在多数实现中，在映射共享的页面时不会实际读入页面的内容，而是在访问页面时，页面才会被每次一页的读入，磁盘文件则被当作后备存储
- 当进程退出或显式地解除文件映射时，所有被修改页面会写回文件

内存映射——mmap()函数

```
void *mmap(void *start, size_t length, int prot, int flags, int fd, off_t offset);
```

- 将指定文件fd中偏移量offset开始的长度为length个字节的一块信息映射到虚拟空间中起始地址为start、长度为length个字节的一块区域
- 得到vm_area_struct结构的信息，并生成相应页表项，建立文件地址和区域之间的映射关系

prot 指定该区域内页面的访问权限位，对应vm_area_struct结构中的vm_prot字段

PROT_EXE: 页面内容由指令组成

PROT_READ: 区域内页面可读

PROT_WRITE: 区域内页面可写

PROT_NONE: 区域内页面不能被访问

内存映射——mmap()函数 (续1)

flags指定所映射的对象的类型，对应vm_area_struct结构中的vm_flags字段

MAP_PRIVATE:

私有的写时拷贝对象，对应可执行文件中只读代码区域（.init、.text、.rodata）和已初始化数据区域（.data）

MAP_SHARED:

共享对象，对应共享库文件中的信息

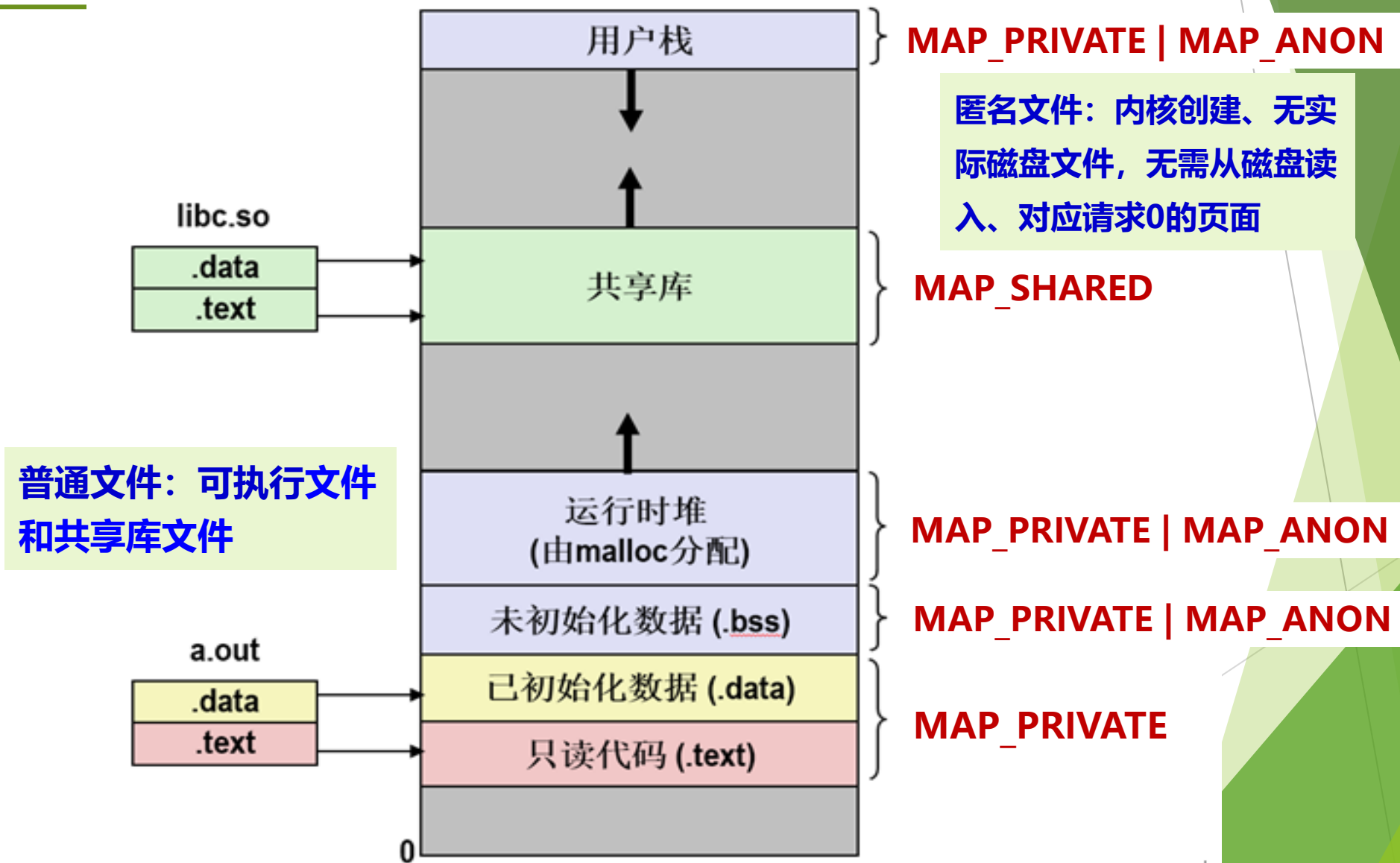
MAP_ANON:

请求0的页，对应内核创建的匿名文件，相应页框用0覆盖并驻留内存

MAP_PRIVATE | MAP_ANON:

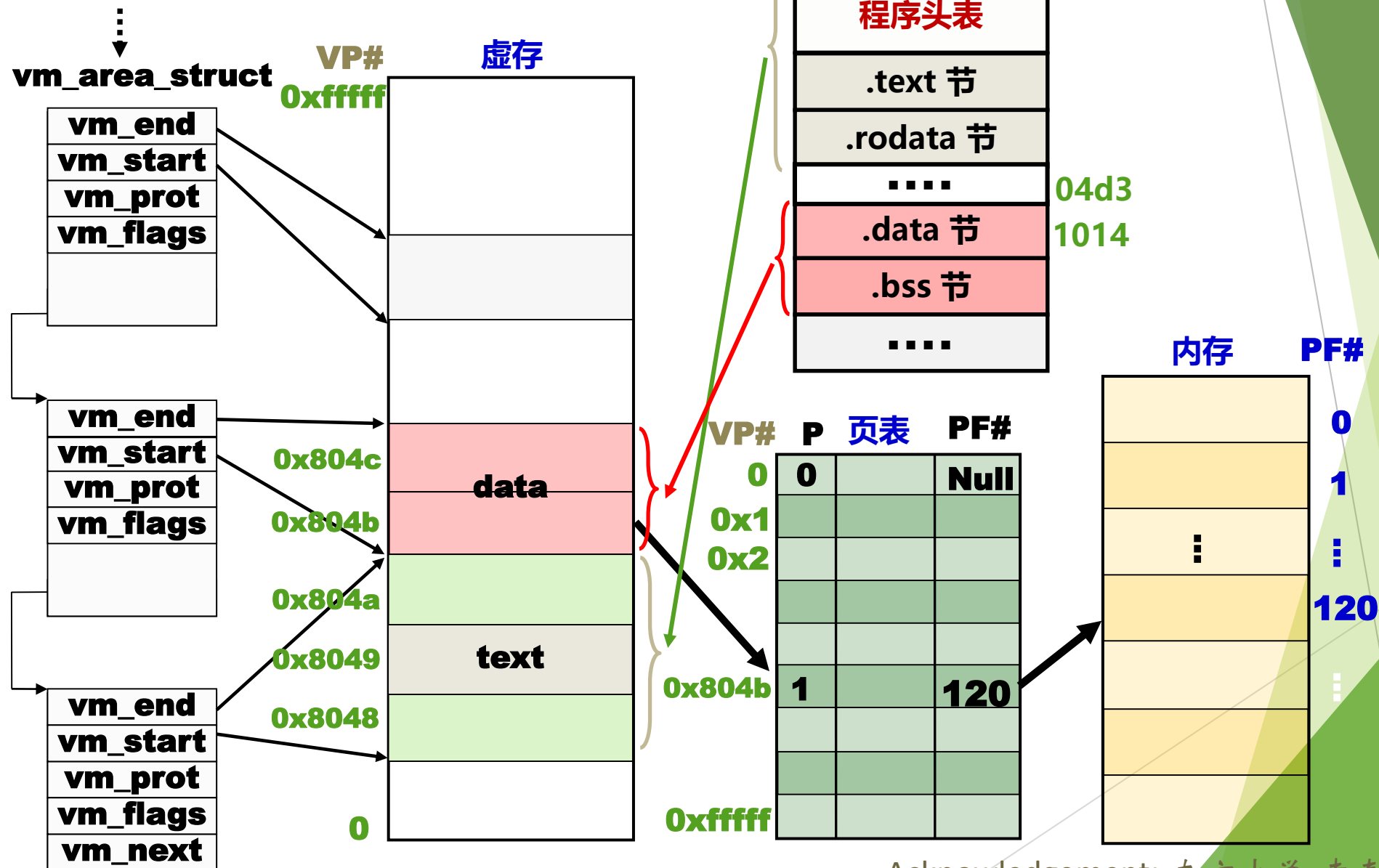
未初始化数据（.bss）、堆和用户栈等对应区域

内存映射——mmap()函数 (续2)



虚拟存储管理全貌

task_struct PCB



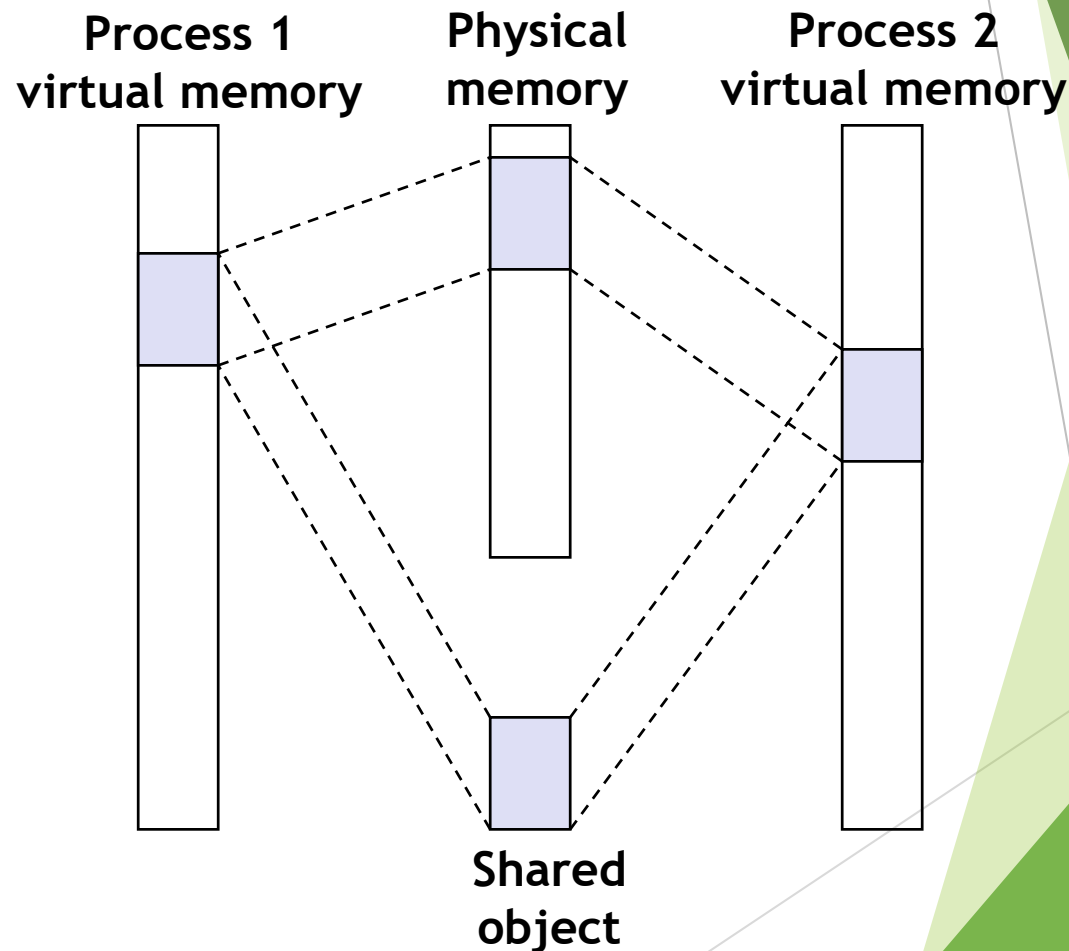
Acknowledgement: 南京大学 袁春风

共享库文件中的共享对象

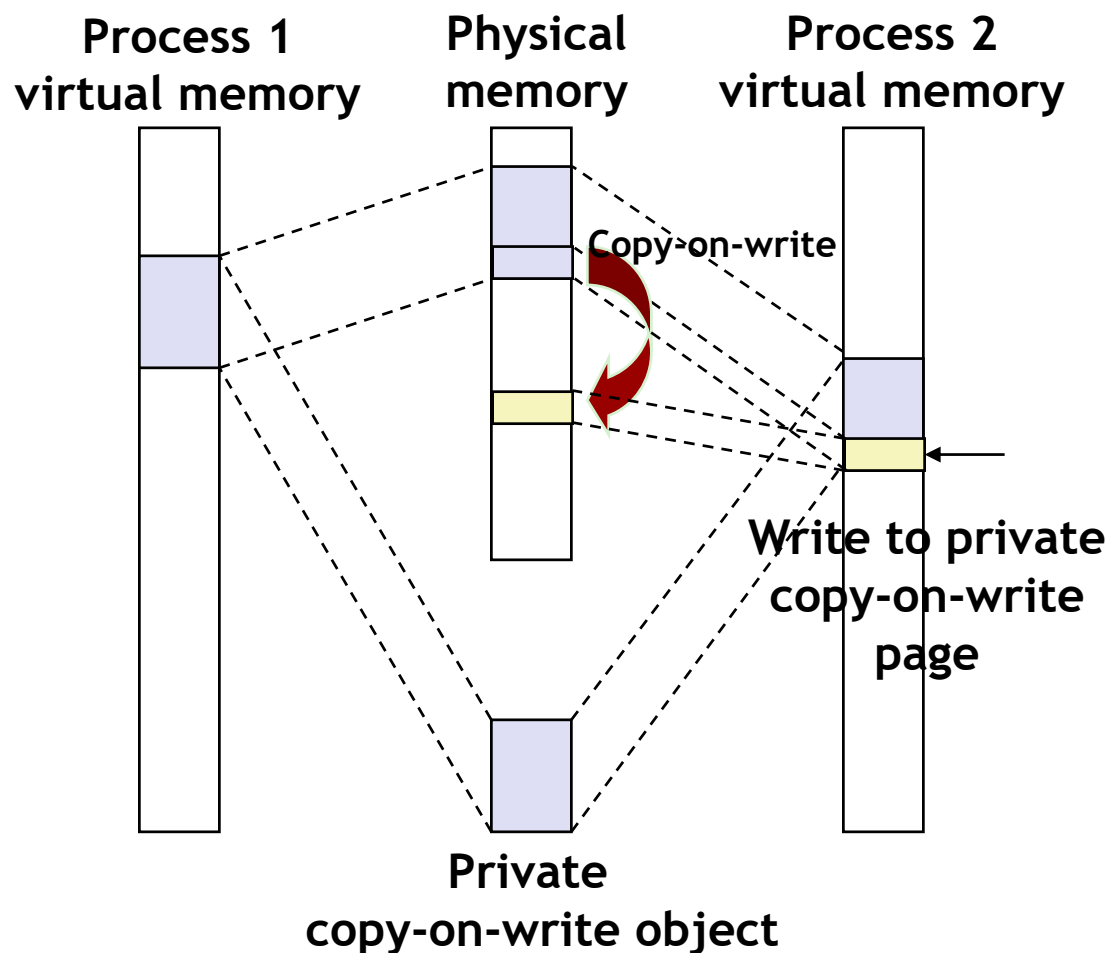
多个进程调用共享库文件中的代码，但共享库代码在内存和硬盘都只需要一个副本

进程1运行过程中，内核为共享对象分配若干页框
进程2运行过程中，内核只将进程2对应区域内页表项中的页框号直接填上即可

一个进程对共享区域进行的操作结果，对于所有进程的共享都是可见的，而且结果也会反映在硬盘上对应的共享对象中



私有的写时拷贝对象



同一个可执行文件对应不同进程时，只读代码区一样，可读可写数据区开始也一样，但属于私有对象

为节省主存，多采用写时拷贝技术

进程1运行过程中，内核为对象分配若干页框，并标记为只读

进程2运行过程中，内核只要将进程2对应区域内页表项中的页框号直接填上，并标记为只读

若两个进程都只是读或执行，则在内存只有一个副本，节省主存；

若进程2进行写操作，则发生访问违例，此时，内核判断异常原因是进程试图写私有的写时拷贝页，就会分配一个新页框，把内容拷贝到新页框，并修改进程2的页表项

回顾：UNIX的fork()实现

- ▶ 为子进程分配一个空闲的进程描述符
proc 结构
- ▶ 分配给子进程唯一标识 pid
- ▶ **以一次一页的方式复制父进程地址空间**
- ▶ 从父进程处继承共享资源，如打开的文件和当前工作目录等
- ▶ 将子进程的状态设为就绪，插入到就绪队列
- ▶ 对子进程返回标识符 0
- ▶ 向父进程返回子进程的 pid

如何
优化？

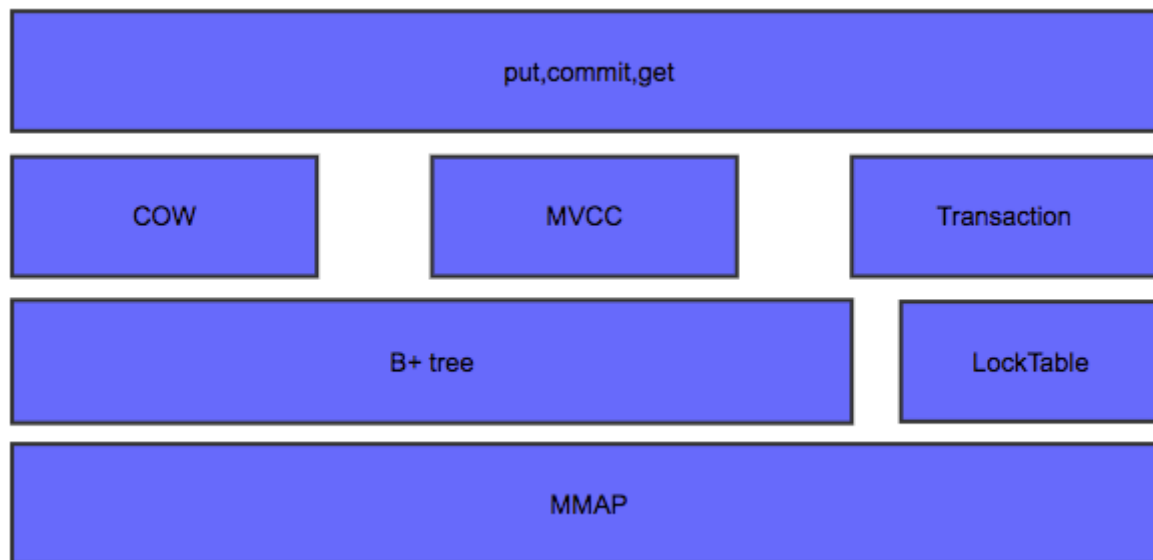
Linux的解决方案是利用
存储管理模块中的“写
时复制技术”COW
(Copy-On-Write) 对
fork()进行了优化

重新审视fork函数

- ▶ VM和内存映射解释了fork如何为每个进程提供私有地址空间
 - ▶ fork()后跟exec()的常见情况的完美方法
- ▶ 为新进程创建虚拟地址空间的步骤
 - ▶ 创建新的进程mm_struct、vm_area_struct、页表的精确副本
 - ▶ 将两个进程中的每个页面标记为只读
 - ▶ 将两个进程中的每个vm_area_struct标记为私有COW
- ▶ 返回时，每个进程都有虚拟内存的精确副本
- ▶ 后续写入使用COW机制创建新页面

内存映射文件的应用

- ▶ LMDB (Lightning Memory-Mapped Database)
- ▶ 是内存映射型数据库，通过使用内存映射文件，可以提供更好的输入/输出性能，对于用于神经网络的大型数据集(比如 ImageNet)，可以将其存储在 LMDB 中



Windows 虚存管理

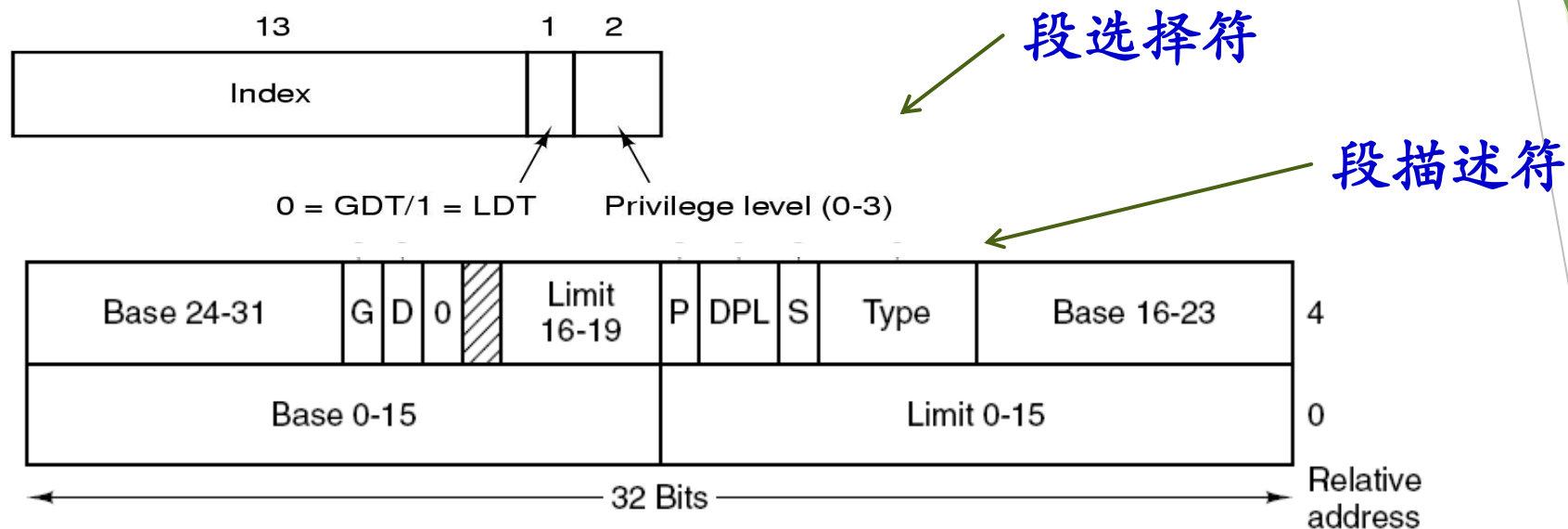
Windows虚拟存储管理

- ▶ Intel x86 虚拟内存机制
 - ▶ 段页式、段描述符、地址转换
- ▶ Windows虚拟内存管理
 - ▶ 内存管理器组、地址空间布局、地址转换机制、缺页处理
 - ▶ 内存分配方式
 - ▶ 工作集
 - ▶ 物理内存管理

Intel x86 虚拟内存机制

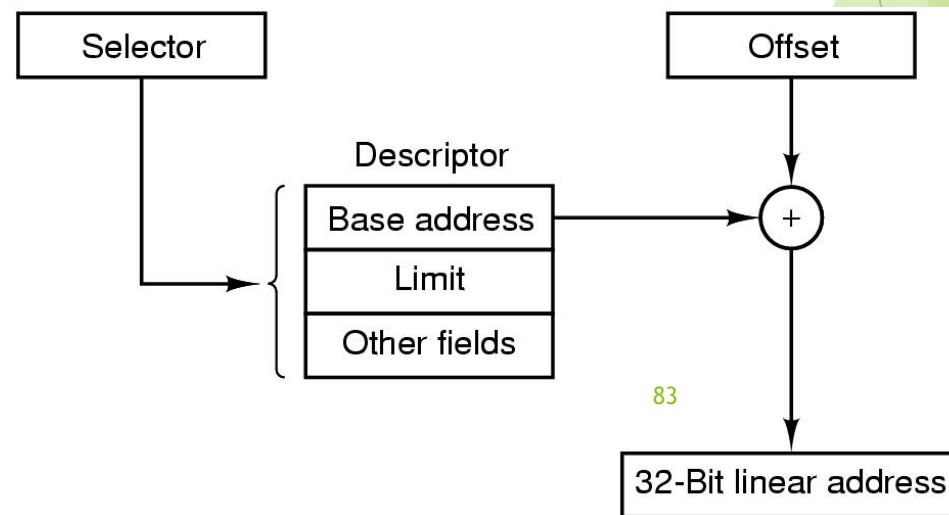
段式寻址、页式及地址转换……

保护模式下的寻址方式



思考:

- ✓ x86提供几种工作模式?
- ✓ 保护模式下的寻址方式?
- ✓ 地址保护如何完成?
- ✓ 如何绕过分段机制?



逻辑地址到物理地址的转换

分页机制
开启

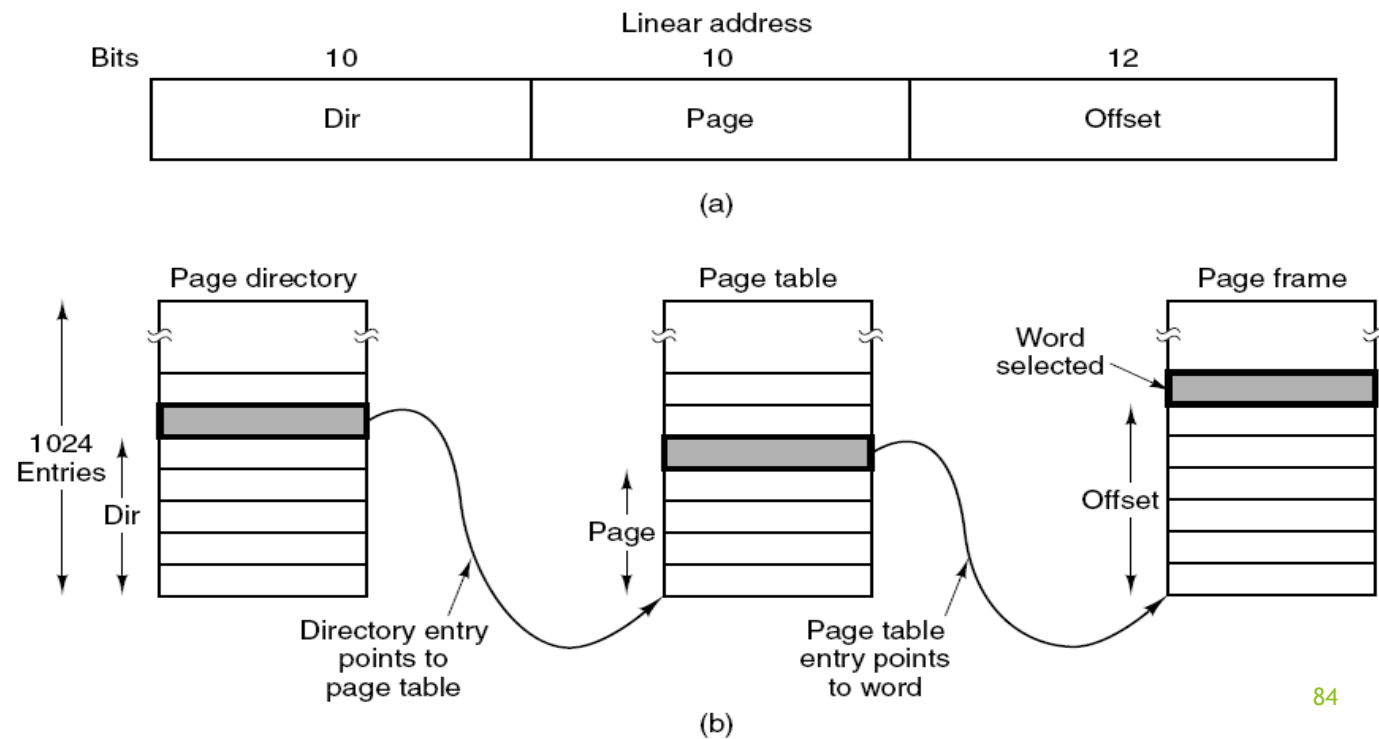


控制寄存器

CR0

CR2

CR3



x86页目录项和页表项

页目录项 PDE (Page Directory Entry)

PFN	Avail	G	P S	0	A	P C D	P W T	U /S	R/ W	P
-----	-------	---	--------	---	---	-------------	-------------	---------	---------	---

页表项 PTE (Page Table Entry)

PFN	Avail	G	0	D	A	P C D	P W T	U /S	R/ W	P
-----	-------	---	---	---	---	-------------	-------------	---------	---------	---

63	62	52	51				12	11	9	8	7	6	5	4	3	2	1	0
N X	AVL	Physical page number				AVL	G	P A T	D	A	P C D	P W T	U / S	R / W				P

AMD x64体系结构中页表项

Core i7 Level 1-3 Page Table Entries

63	62	52	51	12	11	9	8	7	6	5	4	3	2	1	0
XD	Unused	Page table physical base address				Unused	G	PS		A	CD	WT	U/S	R/W	P=1
Available for OS (page table location on disk)															P=0

Each entry references a 4K child page table

P: Child page table present in physical memory (1) or not (0).

R/W: Read-only or read-write access access permission for all reachable pages.

U/S: user or supervisor (kernel) mode access permission for all reachable pages.

WT: Write-through or write-back cache policy for the child page table.

CD: Caching disabled or enabled for the child page table.

A: Reference bit (set by MMU on reads and writes, cleared by software).

PS: Page size either 4 KB or 4 MB (defined for Level 1 PTEs only).

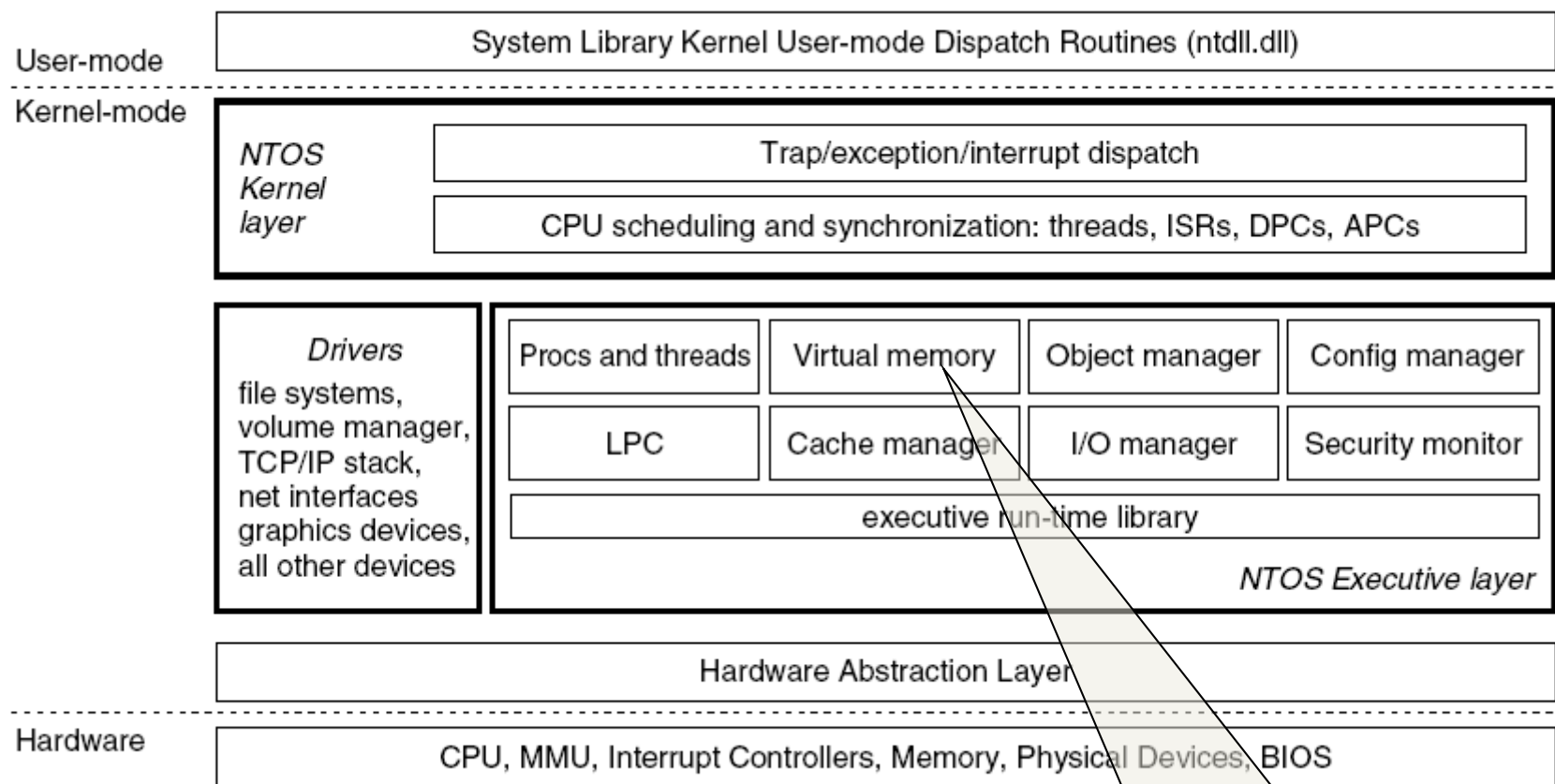
G: Global page (don't evict from TLB on task switch)

Page table physical base address: 40 most significant bits of physical page table address (forces page tables to be 4KB aligned)

*Windows*虚拟内存管理

基本概念、主要方式

Windows的体系结构



内存管理器
位于Ntoskrnl.exe文件中

内存管理器的组成部分

- ▶ 一组执行体系统服务程序

用于虚存的分配、回收和管理。大多数服务是通过Win32 API或内核态的设备驱动程序接口的形式呈现

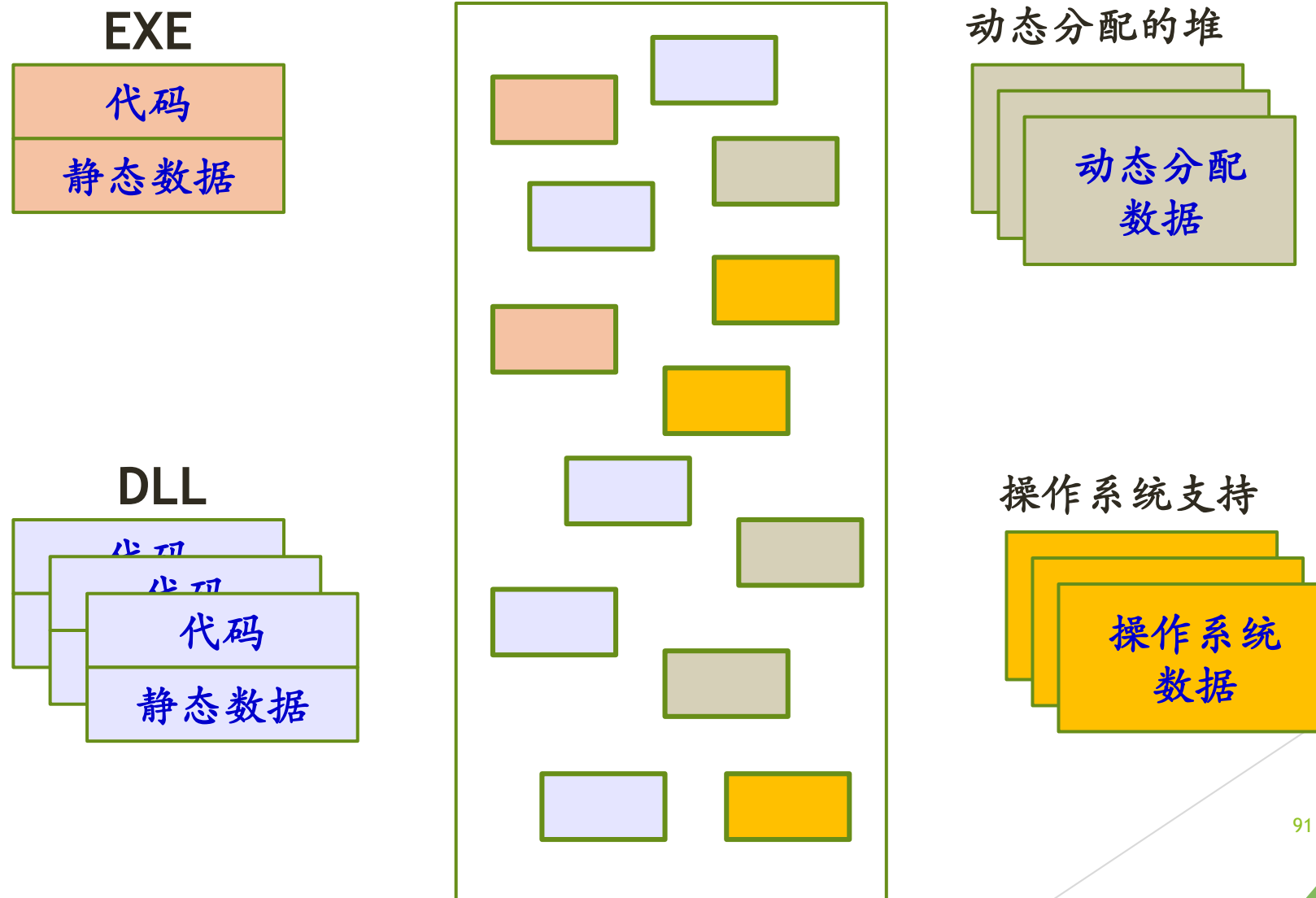
- ▶ 一个页面错误陷阱处理程序

用于解决硬件检测到的内存管理异常
负责分配页框，或把磁盘上的数据装入内存

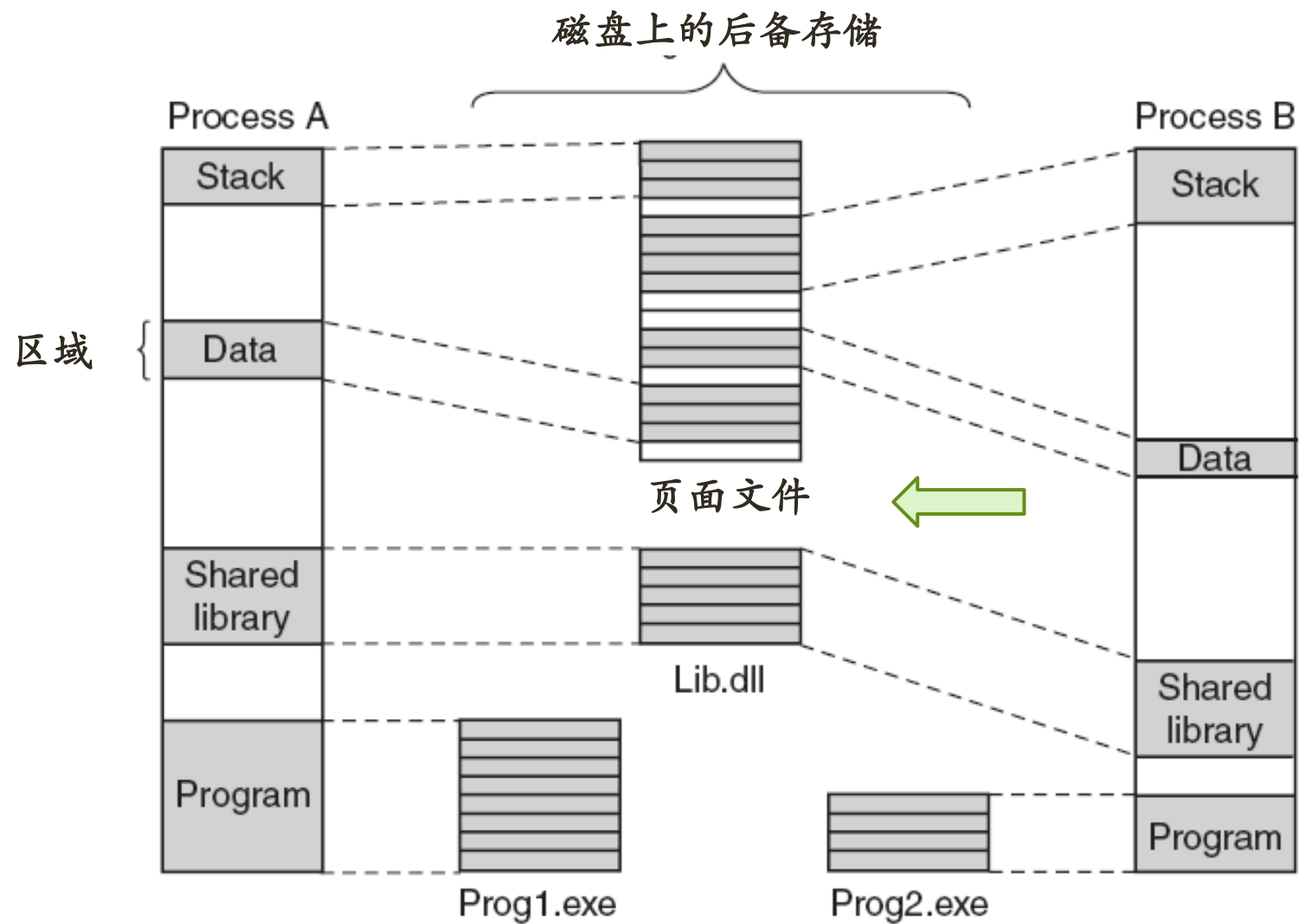
内存管理器的组成部分

- ▶ **工作集管理器(MmWorkingSetManager)**：当空闲内存低于某一界限时，便启动所有的内存管理策略，如：工作集的修整、老化和已修改页面的写入等(16)
- ▶ **进程/栈交换器(KeSwapProcessOrStack)**：(23)
完成进程和内核线程堆栈的换入和换出操作
- ▶ **修改页面写出器(MiModifiedPageWriter)**：(17)
将“脏”页写回到适当的页文件
- ▶ **映射页面写出器(MiMappedPageWriter)**：(17)
将映射文件中的“脏”页写回磁盘
- ▶ **零页线程(MmZeroPageThread)**：(0)
将空闲链表中的页框清零

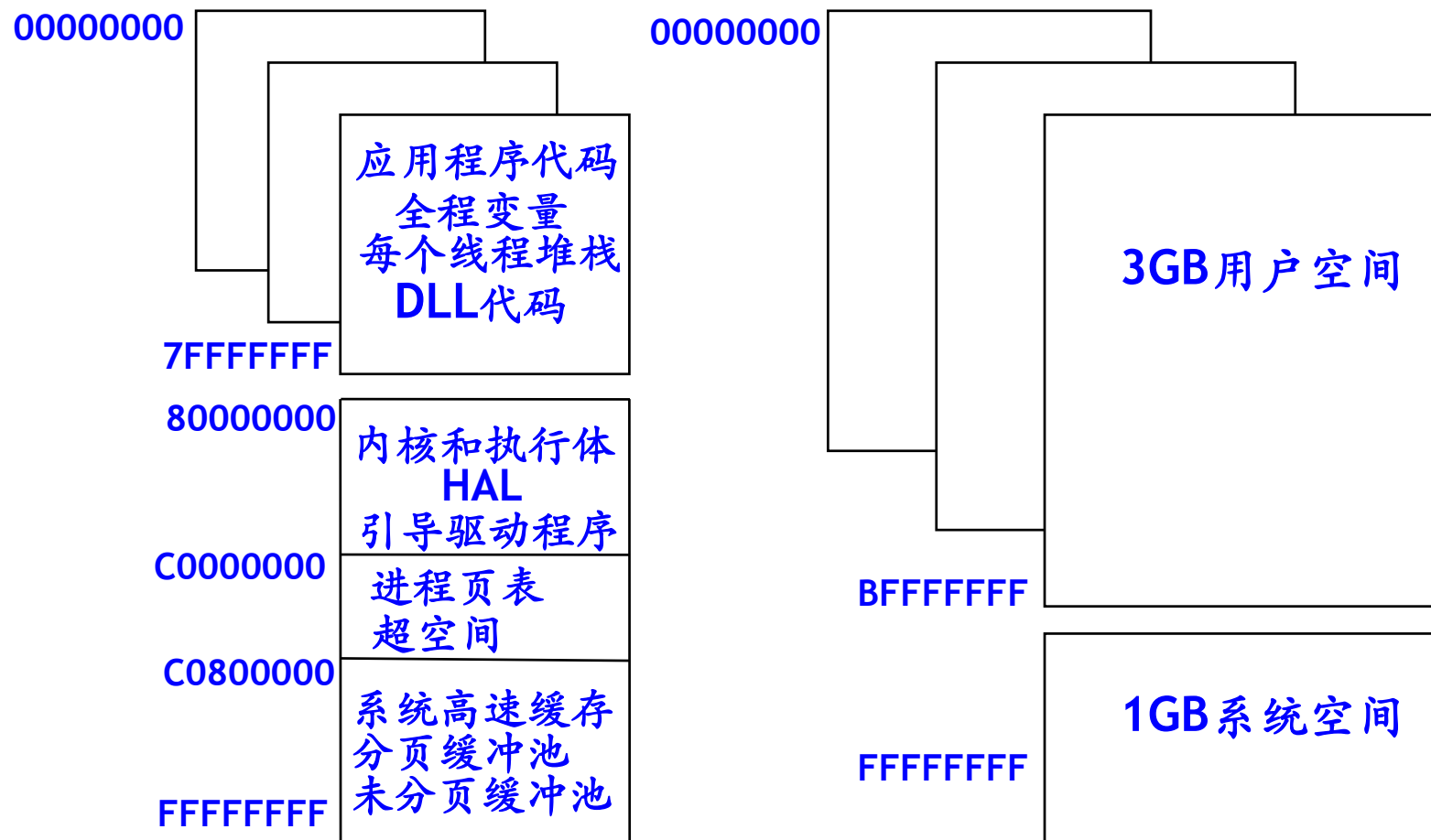
Windows的虚拟内存



磁盘上的相关内容



地址空间布局(1/3)



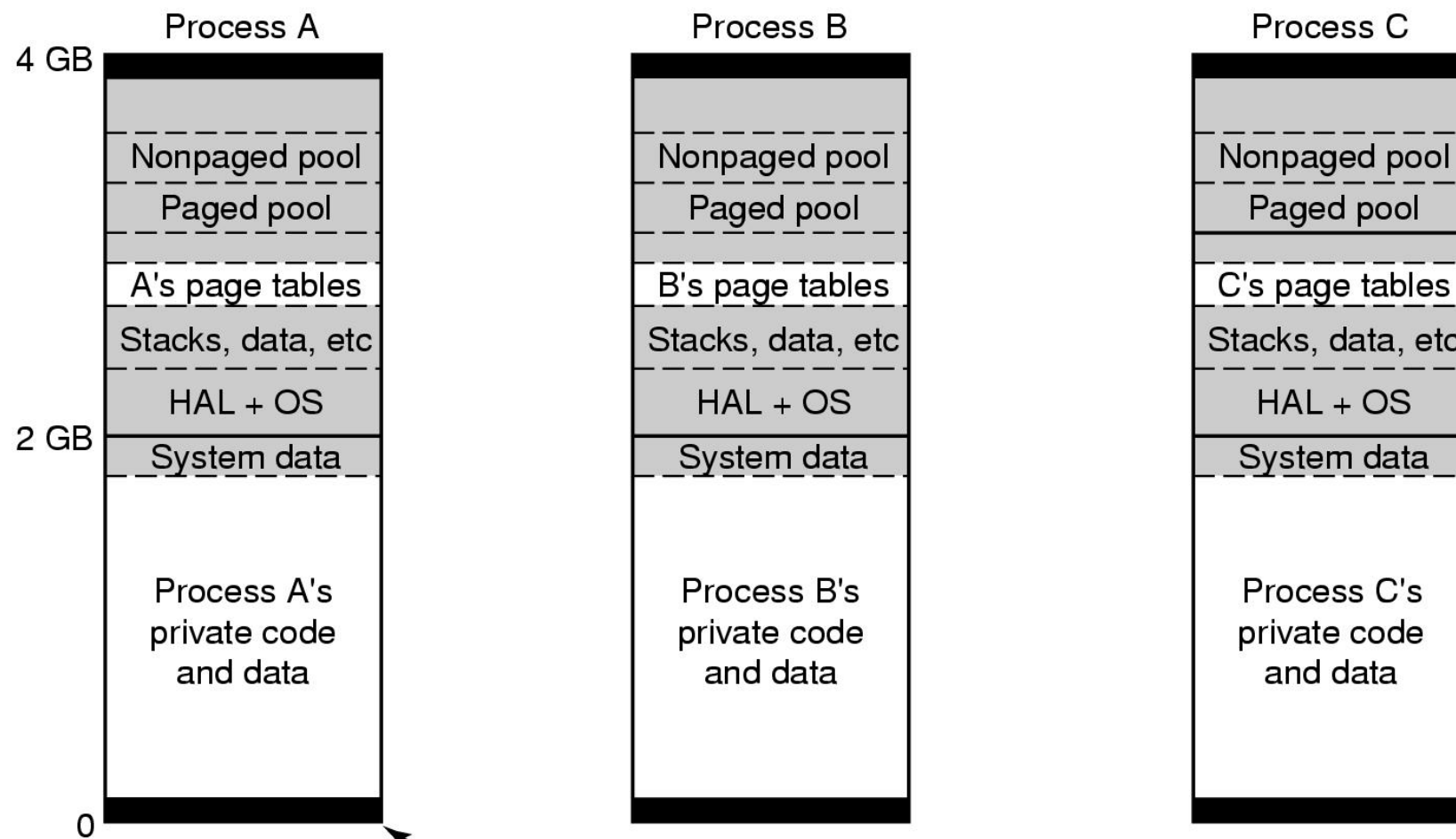
在boot.ini中加入／3GB标志

地址空间布局(2/3)

80000000	系统代码(Ntoskrnl, HAL) 和一些系统中 初始的未分页缓冲池
A0000000	系统映射视图 (例如, Win32k.sys) 或者 会话空间
A4000000	附加的系统PTE (高速缓存可以扩展到 这)
C0000000	进程的页表和页目录 ✓
C0400000	超空间和进程工作集列表
C0800000	没有使用, 不可访问
C0C00000	系统工作集列表
C1000000	系统高速缓存
E1000000	分页缓冲池
EB000000 (min)	系统PTE
	未分页缓冲池扩充
FFBE0000	故障转储信息
FFC00000	HAL使用

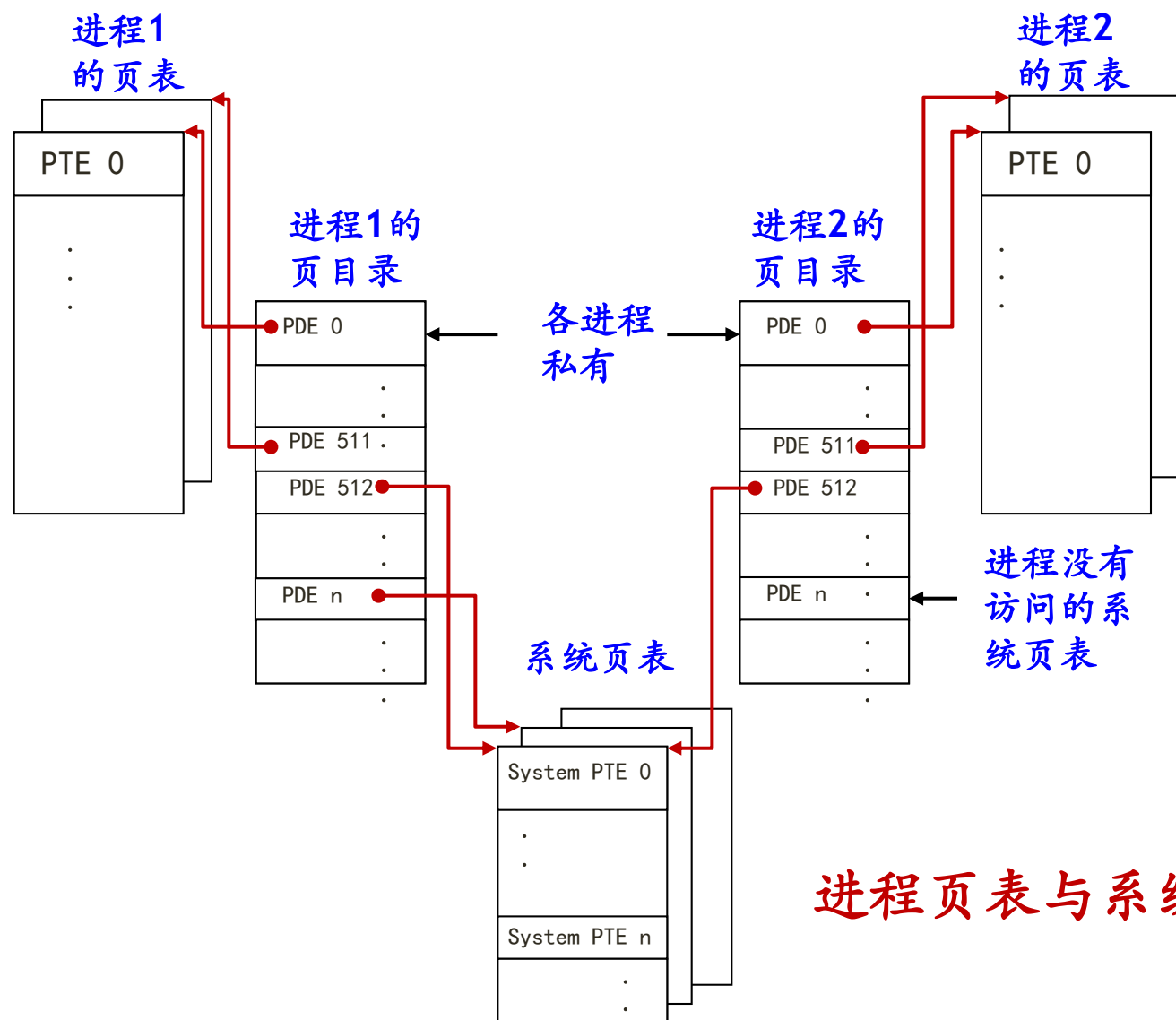
00000000	不可访问
00010000	环境变量 进程的参数 进程堆(Heap) 进程载入的模组 线程栈(Stack) 线程TEB
7FFDE000	第一个线程的线程环境块(TEB)
7FFDF000	进程环境块(PEB)
7FFE0000	共享用户数据页
7FFF0000	不可访问

地址空间布局(3/3)



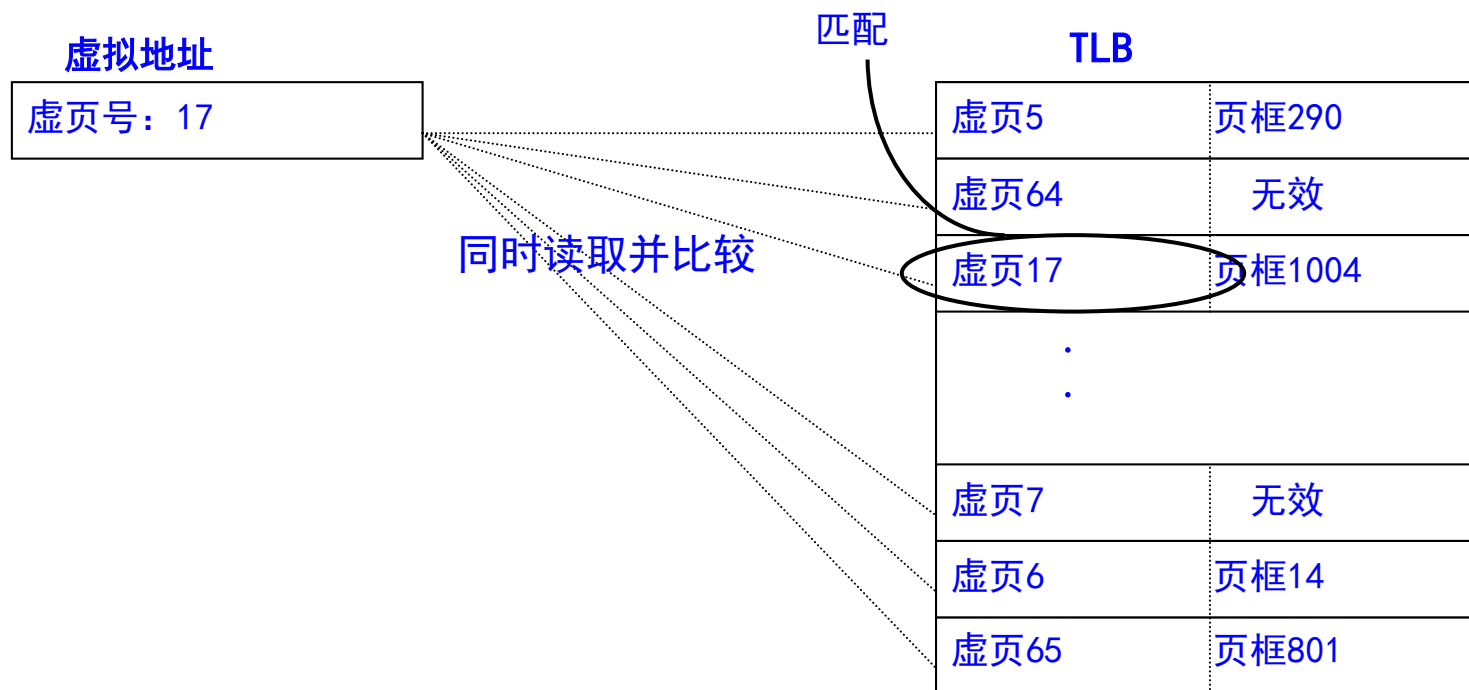
Bottom and top
64 KB are invalid

地址转换机制



进程页表与系统页表

快表TLB



缺页异常处理

*Windows*虚拟内存管理

地址转换

- ▶ 页目录项PDE和页表项PTE的最低位为1，有效(Valid)，表示该页映射了物理内存
- ▶ 当要访问的虚拟页在物理内存中时，该虚拟页对应的 PDE、PTE 都有效，CPU 根据相应的PDE、PTE自动把虚拟地址转换成物理地址，完成访问 —— 这一过程操作系统不需介入
- ▶ 如果要访问的虚拟页不在物理内存，此时的PTE无效（最低位为0）
- ▶ 对无效PTE的格式的定义，由操作系统负责

页面失效——无效的PTE

- ▶ 所引用的页面没有被提交（不在内存）
- ▶ 违反权限的页面访问
- ▶ 修改一个私有共享的写时复制页面
- ▶ 需要扩大栈
- ▶ 所引用的页已被提交但尚未被映射
- ▶ 请求一个零页面

Windows缺页异常处理

- ▶ CPU 会自动通过页目录和页表把虚拟地址 转换成物理地址。当某条指令访问的页不在物理内存中时，在地址转换过程中，CPU将会发现对应地址的页表项无效，从而引发页面错误（Page Fault）异常，该异常的中断号是 0xe，有时也称为缺页中断
- ▶ 发生缺页异常时，CPU 自动将引发异常时访问的虚拟地址存入寄存器 CR2
- ▶ 在发生异常时，CPU 自动根据中断号，在中断描述符表中找到相应的中断描述符，根据中断描述符中的地址，转到异常处理程序 `ntoskrnl!KiTrap0E`
- ▶ 异常处理程序 `KiTrap0E`是操作系统提供的，它将会调用`MmAccessFault`，`MmAccessFault` 通过 CR2 中的访问地址，计算出相应的 PDE / PTE地址，通过分析PTE中的内容（无效的 PDE / PTE的格式由操作系统定义），可以知道是哪种情况引起的异常，并根据情况作出相应的处理

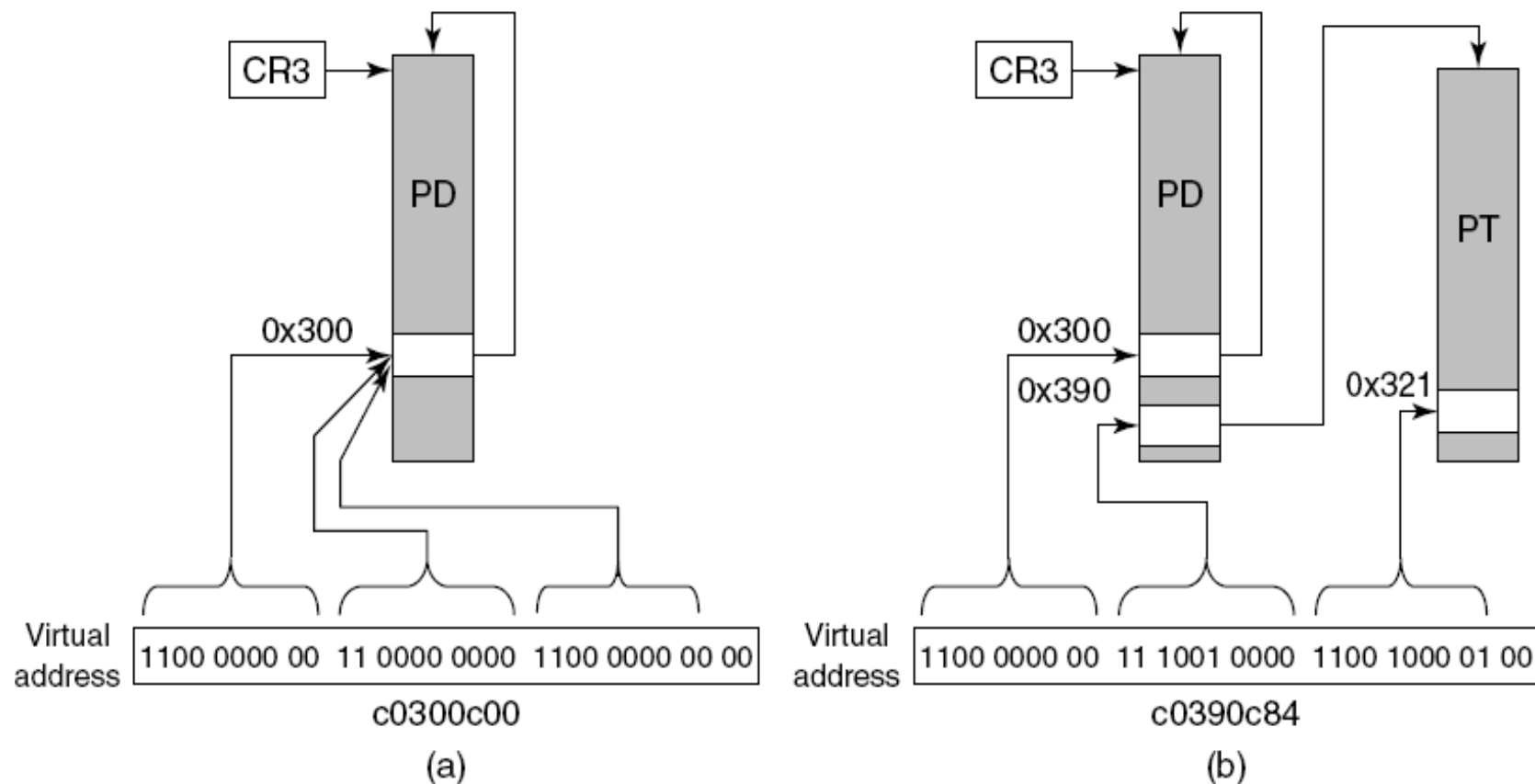
页目录和自映射机制

Windows虚拟内存管理

页目录

- ▶ 内存管理器创建的特殊页，用于映射进程所有页表的位置
- ▶ 其物理地址保存在KPROCESS中
- ▶ 硬件访问页目录、页表和页：通过VPN、PFN完成
- ▶ 内核：通过虚地址来对它们进行访问
 - x86中，它还同时被映射到地址**0xC0300000**处
- ▶ 专用寄存器（x86中为CR3）用于保存页目录的物理地址

页目录自映射机制



自映射: $PD[0xc0300000 \gg 22]$ 即 PD

虚拟地址(a): $(PTE *)(0xc0300c00)$ 指向 $PD[0x300]$, 即自映射页目录项

虚拟地址(b): $(PTE *)(0xc0390c84)$ 指向 PTE, 对应虚拟地址 `0xe4321000`

页目录自映射机制(续)

宏MiGetPdeAddress():

给定一个虚拟地址va，计算出对应的PDE

```
((PMMPTE)((((ULONG)(va))>>22)<< 2) + PDE_BASE))
```

宏MiGetPteAddress():

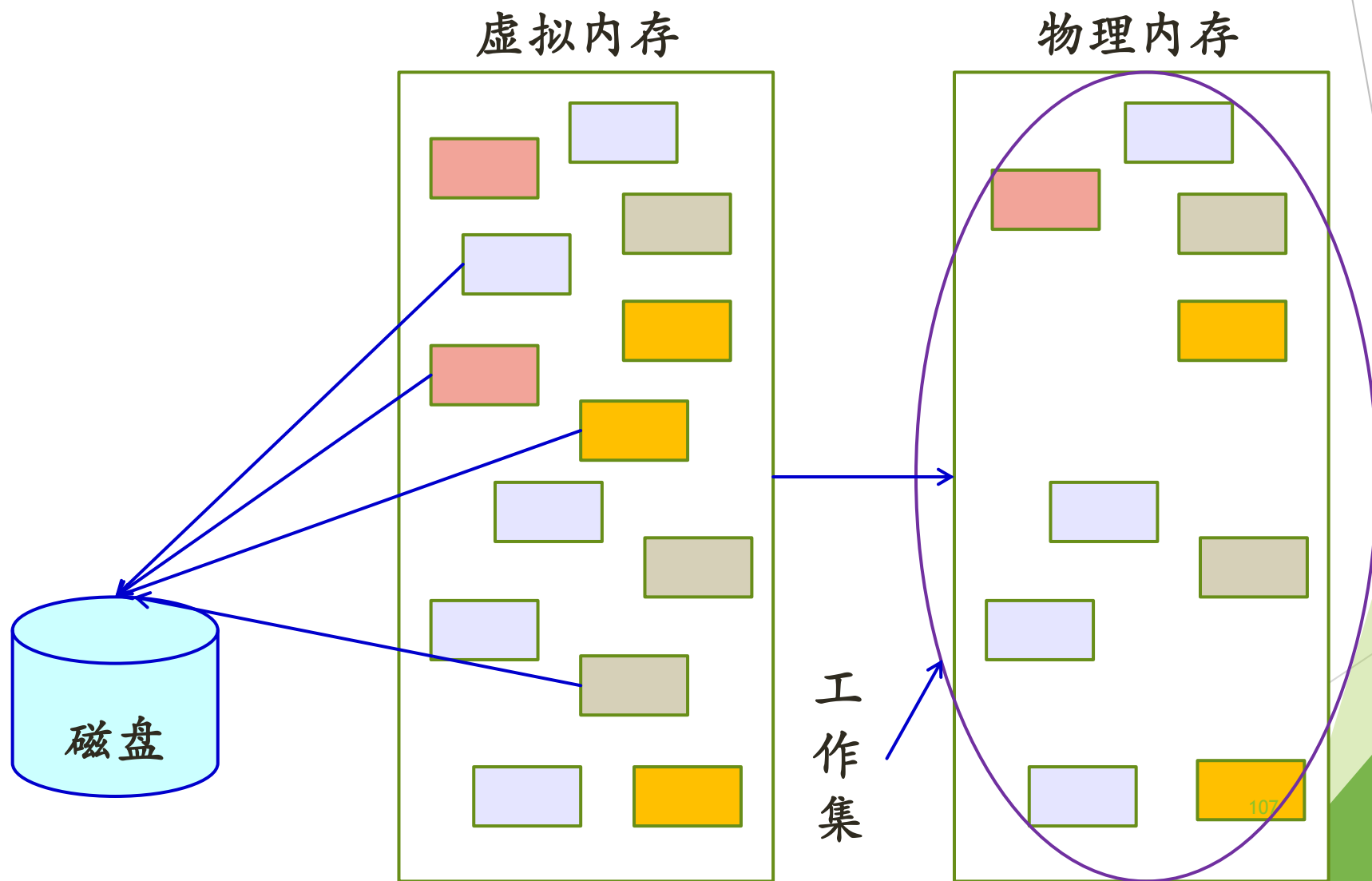
给定一个虚拟地址va，计算出对应的PTE

```
((PMMPTE)((((ULONG)(va))>>12)<< 2) + PTE_BASE))
```

工作集

*Windows*虚拟内存管理

Windows的工作集(1/3)



Windows的工作集(2/3)

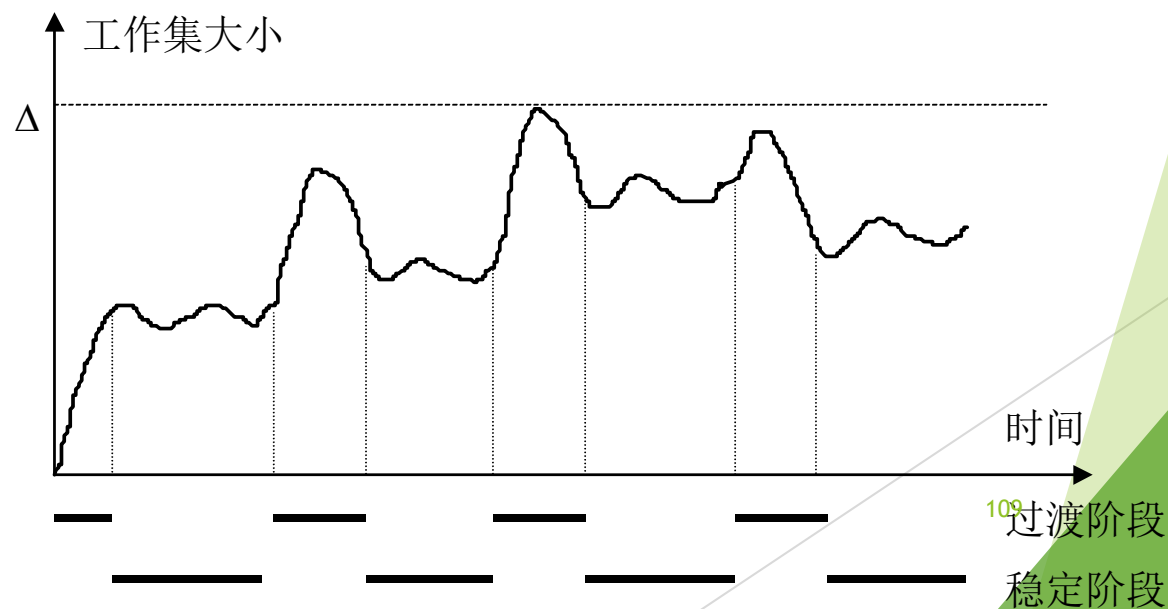
► 工作集

驻留在物理内存中的虚拟页面的子集

- ✓ 进程工作集：为每个进程分配的一定数量的页框
- ✓ 系统工作集：为可分页的系统代码和数据分配的页框
- 内存管理器利用请求式页面调度算法以及簇方式将页面装入内存
- 置页策略：选择页框应使CPU内存高速缓存不必要的震荡最小

Windows的工作集(3/3)

- ▶ 工作集大小的变化：进程开始执行后，随着访问新页面逐步建立较稳定的工作集
- ▶ 当内存访问的局部性区域的位置大致稳定时，工作集大小也大致稳定
- ▶ 局部性区域的位置改变时，工作集快速扩张和收缩过渡到下一个稳定值



页面置换算法

► 基于工作集模型

- 由已装入内存的页框组成
- 最小值/最大值
- 当可用页框数量降低到一定程度时，启动工作集修整策略

动态改变大小和构成

平衡集管理器线程调用工作集管理器

► 周期性检查

大量可用内存、内存开始紧张、内存紧缺

用户空间内存分配方式

Windows虚拟内存管理

用户空间内存分配方式

- ▶ 以页为单位的虚拟内存分配方式
函数 (Virtualxxx)
- ▶ 内存映射文件
函数 (CreateFileMapping, MapViewOfFile)
- ▶ 内存堆方法
(Heapxxx 和早期的接口 Localxxx 和 Globalxxx)

以页为单位的虚拟内存分配(1/3)

- ▶ 进程地址空间0x0~0x7FFFFFFF，用户程序必须经过“**保留**”和“**提交**”两个阶段使用一段地址范围
- ▶ VirtualAlloc和VirtualAllocEx函数实现这些功能，用户程序可以首先保留地址空间，然后向此地址空间提交物理页面
- ▶ 保留地址空间是为线程将来使用所保留的一块虚拟地址。在已保留的区域中，提交页面必须指出将物理存储器提交到何处以及提交多少；提交页面在访问时会转变为物理内存中的有效页面
- ▶ VirtualFree或VirtualFreeEx函数回收页面或释放地址空间

以页为单位的虚拟内存分配(2/3)

问题？

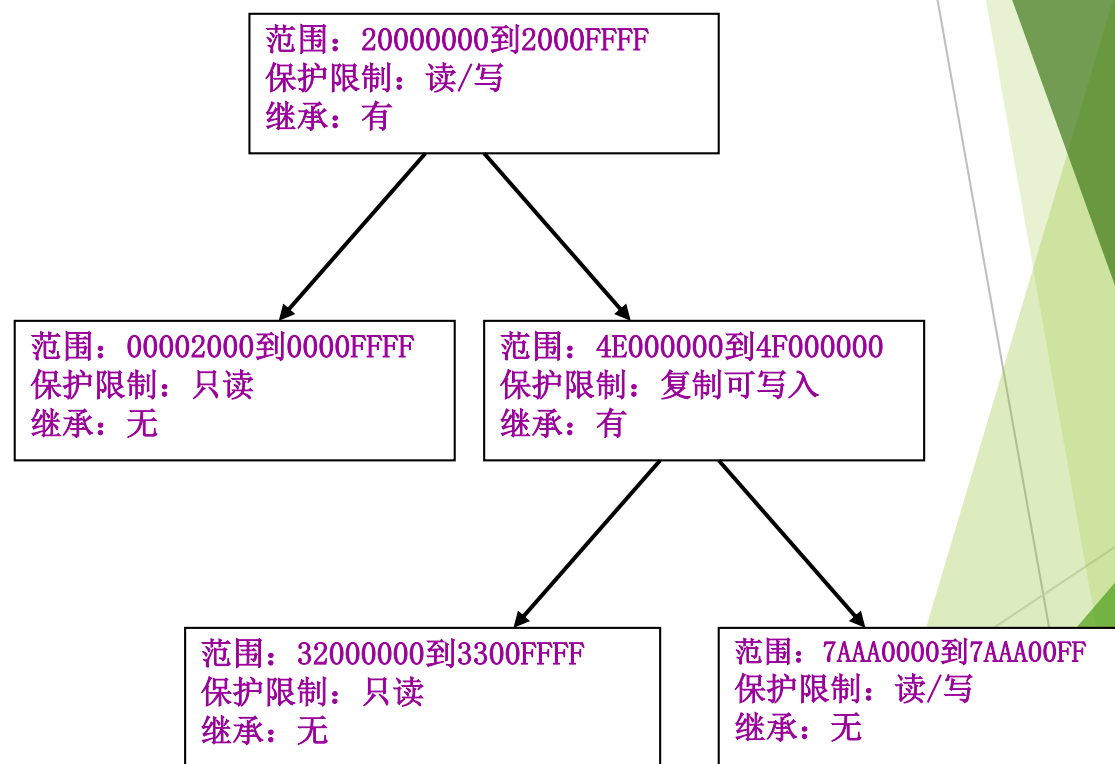
- ▶ 使用 VirtualAlloc 可以在用户地址空间中保留或者提交指定地址和大小的一段地址空间，那么系统如何知道指定的这段地址空间是不是已经被分配（保留或者提交）？
- ▶ 对于指定地址空间是否已经被提交了物理内存，可以通过页目录和页表来判断，不过这样做很麻烦；而对于指定地址空间是否已经被保留，通过页目录和页表是没有办法判断的
- ▶ Windows中使用 VAD 来解决这个问题

以页为单位的虚拟内存分配(3/3)

对于每一个进程，Windows 的内存管理器维护一组虚拟地址描述符（VAD）来描述一段被分配的进程虚拟空间的状态

虚拟地址描述信息被构造成一棵自平衡二叉树以使查找更有效率

虚拟地址描述符VAD



VAD二叉排序树的根的地址保存在进程结构 EPROCESS 中

利用区域对象实现内存映射文件

通过区域对象服务提供的大数据流和内存共享服务

- 区域对象（section object）在Win32子系统中称为文件映射对象
- 表示可以被两个或更多进程所共享的内存块，区域对象也可以基于页文件
 - ◆ CreateFileMapping创建区域对象
 - ◆ MapViewOfFile函数映射区域对象的一部分（叫做区域视图），并指定映射范围
- 当两个进程对同一区域对象建立视图时，就发生了对该区域对象的共享

利用区域对象实现内存映射文件

- ▶ 使用区域对象将一个文件映射到进程的地址空间，之后访问这个文件就象访问内存中的一个大数据组，而不是对文件进行读写
- ▶ 当程序访问一个无效的页面（不在物理内存中的页面）时，引起缺页中断，内存管理器会自动将这个页面从映射文件调入内存
- ▶ 如果应用程序修改了页面，内存管理器在页面进行常规调度时将修改写回文件（或者应用程序可以通过利用Win32的FlushViewOfFile函数来刷新一个视图）

内存堆方法

- ▶ 内存堆方法适合于大量的小型内存申请
- ▶ 堆(heap)是保留的地址空间中一个或多个页组成的区域，这个地址区域可以由堆管理器按更小块划分和分配
- ▶ 堆管理器是执行体中分配和回收可变内存的函数集
- ▶ 进程启动时带有一个缺省进程堆，通常是1MB大小。为了从缺省堆中分配内存，线程必须调用GetProcessHeap函数得到一个指向它的句柄。有了句柄后，线程可以调用HeapAlloc和HeapFree来从堆中分配和回收内存块
- ▶ 进程也可以使用HeapCreate函数创建另外的私有堆。当进程不再需要私有堆时，可以通过调用HeapDestroy释放虚拟地址空间

物理内存管理

*Windows*虚拟内存管理

清除策略要点

- 分页系统工作的**最佳状态**：发生缺页异常时，系统中有大量的空闲页框
- **结论**：保存一定数目的页框供给比使用所有内存并在需要时搜索一个页框有更好的性能
- 当需要使用一个已置换出的页框时，如果该页框还没有被新内容覆盖，则将它从空闲页框缓冲池中移出即可恢复该页面

物理内存管理

- ▶ 页框号数据库（ PFN数据库 ）
- ▶ 结构MMPFN（24字节）
保存每一个物理页的相关信息
- ▶ 全局变量 MmPfnDatabase
保存页框号数据库的首地址

页框的状态(1/2)

- ▶ **活动 (Active) /有效 (Valid)**

该页框在某个进程的工作集中

此进程的对应页表项是有效的，从高20bit可获得PFN

- ▶ **过渡 (Transition)**

系统正在从一个文件将内容读入该页框，或者正在向一个文件写出该页框的内容

- ▶ **空闲 (Free)**

该页框中的内容不再被需要

- ▶ **零初始化 (zeroed)**

该页框空闲并且已经被用零初始化

- ▶ **坏 (Bad)**

该页框存在硬件错误，不能被使用

页框的状态(2/2)

□ 后备(standby)

- ▶ 该页框曾经在某个进程的工作集中，且该页框的内容在被此进程使用时没有改变
- ▶ 该页框现在已被移出该进程的工作集，但页框中的内容仍是此进程的内容，即对应的PTE中的高20bit仍然是该页框号，只是该PTE被标记为 **invalid** 和 **transition**
- ▶ 当此进程需要再次访问这一页内容时，只需要重新设定该PTE的标志，并把该PTE变为有效，即把该页框从 **standby** 状态变为 **active(valid)** 状态即可

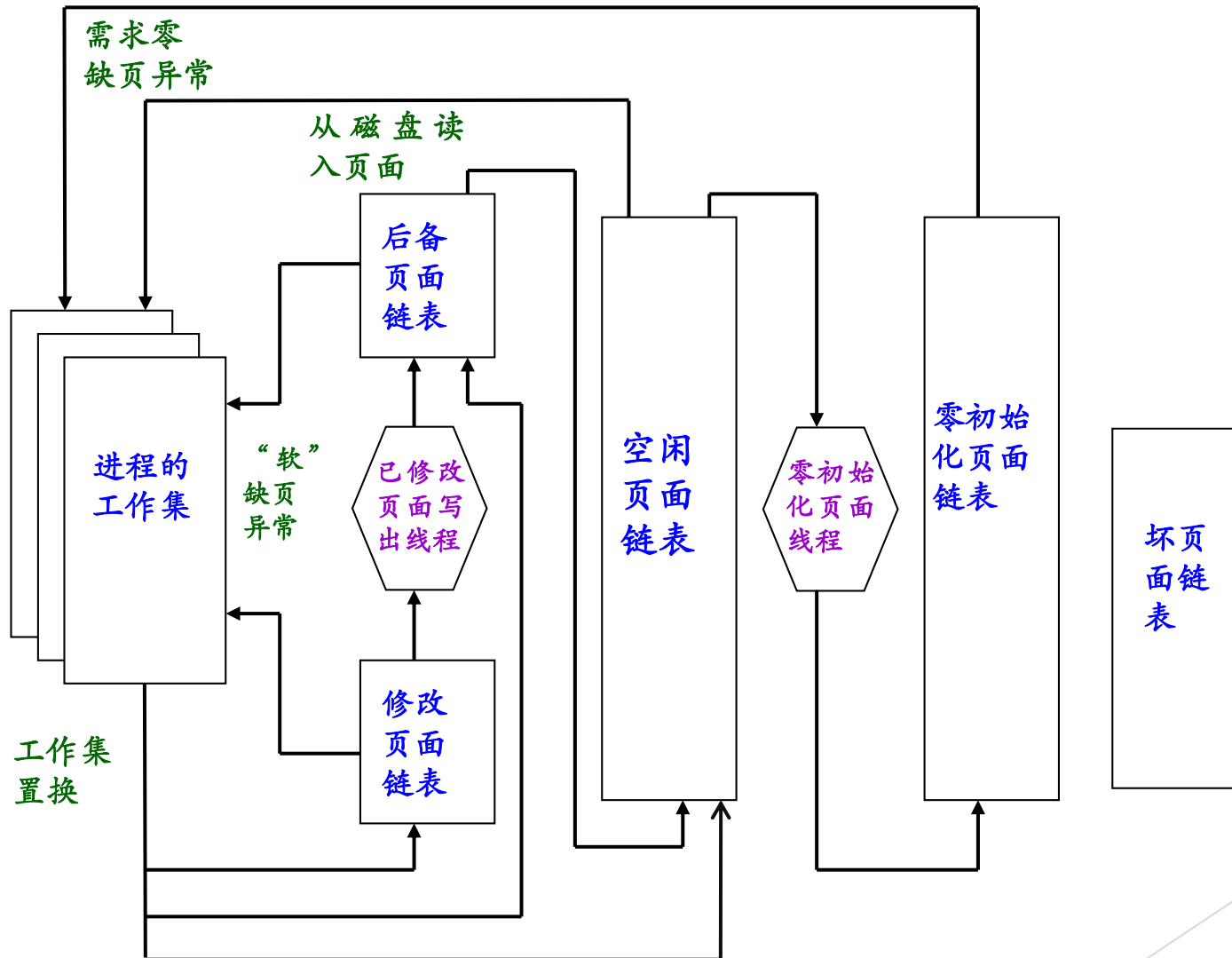
□ 修改 (Modified)

- ▶ 该页框曾经在某个进程的工作集中，且该页框的内容在被此进程使用时有改变
- ▶ 该页框现在已被移出该进程的工作集，但页框中的内容仍是在此进程的内容，即对应的PTE中的高20bit仍然是该页框号，只是该PTE被标记为 **invalid** 和 **transition**
- ▶ 当此进程需要再次访问这一页内容时，只需要重新设定该PTE的标志，并把该PTE变为有效，即把该页框从 **modified** 状态变为 **active(valid)** 状态就可以了
- ▶ 在该页框被系统作为其他用途使用之前，需要将该页框中的内容写入硬盘中的页文件中

页表与页框号数据库

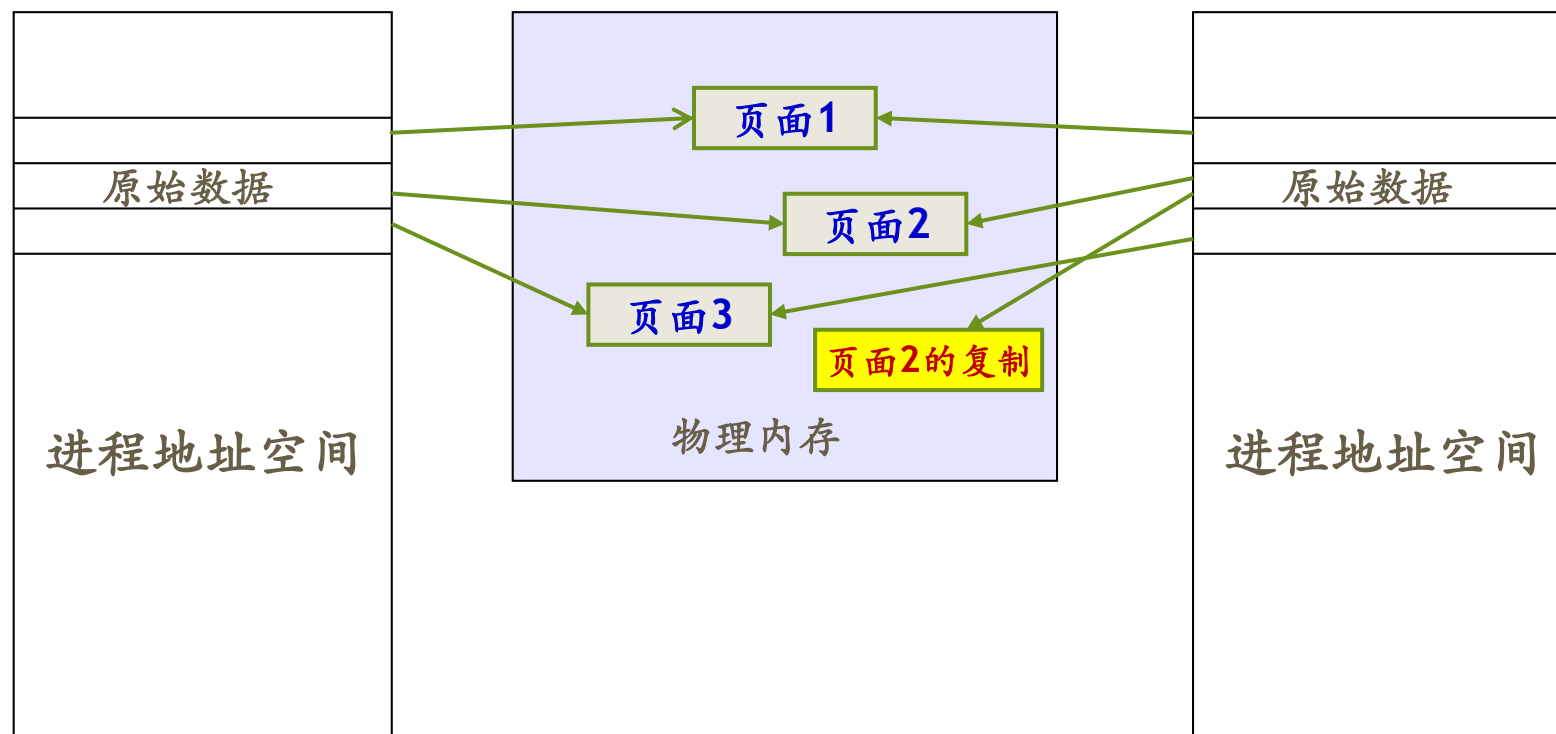


页框数据库实现与应用



支持写时复制技术

进程试图改变
页面2的数据后



例如，两个进程共享三个页，每页个都标志成写时复制

新复制的页面对执行写操作的进程是私有的，对其他共享写时复制页面的进程是不可见的

重点小结

□ 基本概念

- ▶ 虚拟内存、虚拟地址、虚拟地址空间
- ▶ 页表表项、多级页表、反转页表（倒排页表）
- ▶ 快表TLB

□ 虚拟内存管理

- ▶ 取页策略（按需、预取）
- ▶ 驻留集管理（固定、可变）
- ▶ 置换范围与策略（局部、全局）
- ▶ 清除策略（页缓冲）
- ▶ 加载控制

□ 内存映射文件、写时复制

源代码阅读

► XV6源代码阅读——虚拟存储管理

阅读并理解下列文件中的代码：

kernel/memorylayout.h, kernel/vm.c, kernel/kalloc.c,
kernel/exec.c, kernel/elf.h

可参考XV6手册 §3.1-§3.8的内容

Many Thanks

The End